FELIX JOHANN STECK

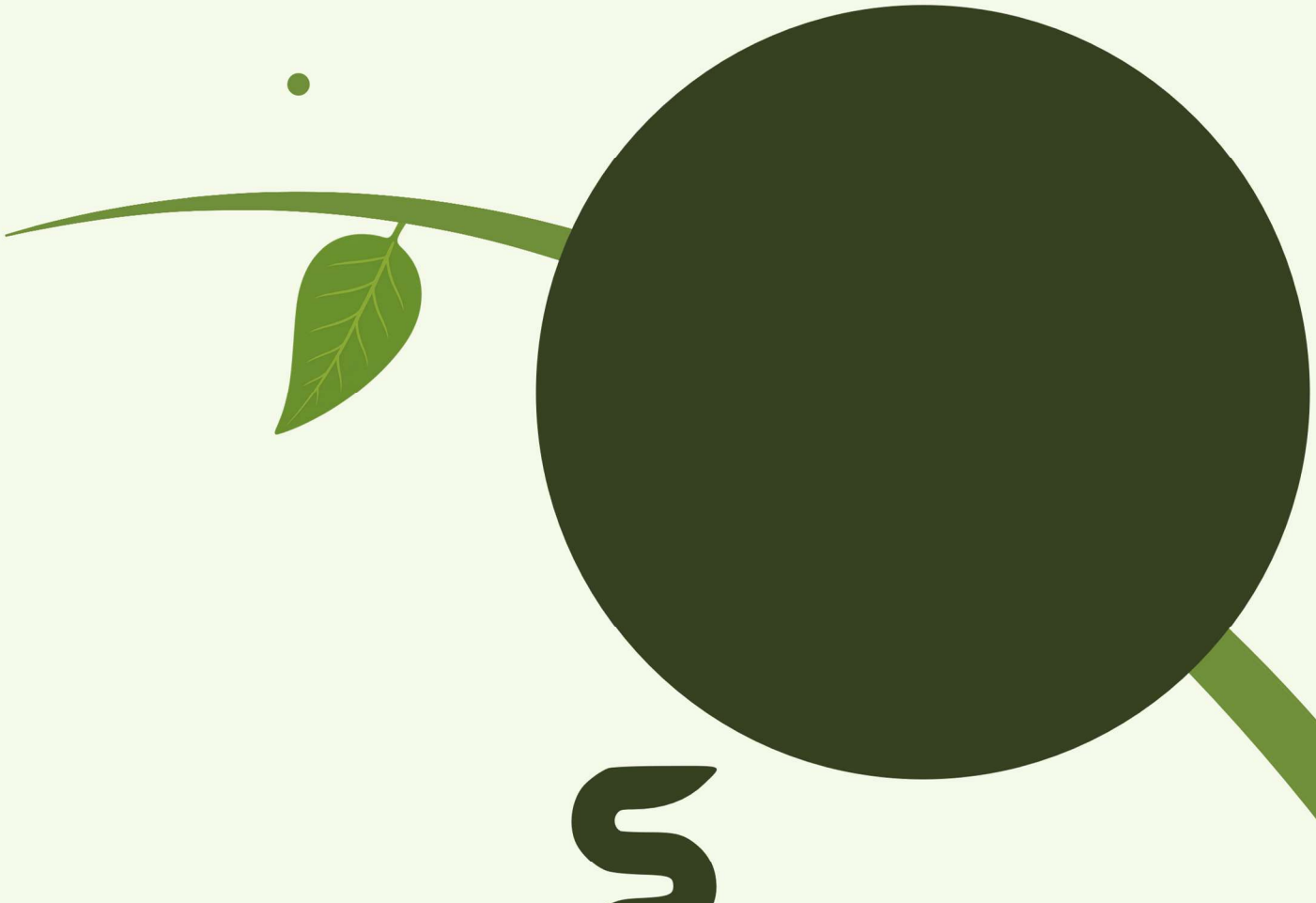
SCHULJAHR 2024/25

5B ELEKTRONIK

TFO MERAN

# PROJEKTDOKUMENTATION

# EIC



STECKTEC

# Inhaltsverzeichnis

**1. Vorwort ..... 4**

1.1 Was ist Stecktec EIC ..... 4

1.2 Idee / Inspiration ..... 4

1.3 Was bedeutet EIC ..... 4

1.4 Anwendungs-/Erweiterungsmöglichkeiten ..... 5

1.5 Abstract Italiano ..... 5

1.6 Abstract English ..... 5

**2 Gesamtkonzept ..... 6**

2.1 Blockschaltbild ..... 6

**3 Hardware-Grundlagen ..... 7**

3.01 Mikrocontroller ..... 7

3.02 Sim-Modul ..... 9

3.03 Lora-Modul ..... 10

3.04 BME280 Sensor ..... 11

3.05 Watermark Sensor ..... 11

3.06 H-Brücke und Magnetventil ..... 12

3.07 Port Expander ..... 13

3.08 Li-Ion Akku ..... 14

3.09 Solarzelle ..... 14

3.10 Durchflusssensor ..... 15

3.11 Analogmultiplexer ..... 15

3.12 DCDC-Wandler ..... 16

3.13 TP5400 Laderegler ..... 16

**4 Software-Grundlagen ..... 17**

4.1 I2C ..... 17

4.2 Bootloader ..... 17

4.3 Datenbank Firebase ..... 18

4.4 React Native ..... 19

**5 Hardware ..... 20**

5.1 Hauptplatine (V1/V2) ..... 20

5.1.1 Schaltungsbeschreibung ..... 20
5.2 Sensorplatine ..... 26
5.3 Magnetventilsteuerung Platine ..... 29
5.4 Solarversorgungsplatine (V1/V2) ..... 31
5.4.1 Solarversorgungsplatine V1 ..... 31
5.4.2 Solarversorgungsplatine V2 ..... 33
5.5 Programmiergerät ..... 34
5.6 Platinen 3D-Modelle ..... 36
**6 Software ..... 37**
6.1 Ventilansteuerung ..... 38
6.2 Sim-Kommunikation ..... 39
6.2.1 Aufbau einer HTTP-Request ..... 40
6.2.2 Entwicklung und Herausforderungen ..... 41
6.3 Lora Kommunikation ..... 41
6.4 Smartphone App ..... 43
**7 Projektmanagement ..... 45**
7.1 Projektantrag ..... 46
7.2 Pflichtenheft ..... 48
7.3 Beschreibung der Working Packages ..... 49
7.4 Ganttdiagramm ..... 50
7.5 Wochenberichte ..... 54
7.6 Kostenrechnung ..... 56
7.6.1 Beraterstunden ..... 56
7.6.2 Arbeitsstunden ..... 57
7.6.3 Platinenkosten ..... 58
7.6.4 Bauteilkosten LCSC ..... 58
7.6.5 Kosten Module ..... 59
7.6.6 Gesamtkosten Projekt ..... 59
7.8.2 Kosten Serienfertigung ..... 60
**8 Fazit ..... 61**
**9 Abbildungsverzeichniss ..... 62**
**10 Copyright ..... 63**

# 1 Vorwort

## 1.1 Was ist Stecktec EIC

Stecktec EIC ist ein modulares intelligentes Bewässerungssystem, das speziell für den Einsatz im Obst und Beerenanbau, aber auch für private Gärten konzipiert wurde. Es besteht aus mehreren modularen Teilen und kann verschiedene Werte wie die Bodenfeuchtigkeit Temperatur und Luftfeuchtigkeit messen und Bewässerungsventile schalten. Alles wird über eine benutzerfreundliche Smartphone App gesteuert und überwacht. Das ermöglicht dem Bauern eine optimale Übersicht und Kontrolle über seine Felder.

## 1.2 Idee / Inspiration

Die Idee zum EIC-Bewässerungssystem entstand aus der Beobachtung, dass viele bestehende landwirtschaftliche Bewässerungslösungen entweder zu teuer, zu kompliziert oder unzureichend automatisiert sind. Insbesondere kleinere landwirtschaftliche Betriebe oder Privatpersonen verfügen häufig nicht über die Mittel, um teure Systeme mit vielen unnötigen Funktionen zu implementieren. Ziel war es daher, ein kostengünstiges, modulares und leicht erweiterbares System zu entwickeln, das zuverlässig arbeitet und sich an verschiedene Bedürfnisse anpassen lässt.

Ein weiterer wichtiger Beweggrund war die zunehmende Regulierung des landwirtschaftlichen Wasserverbrauchs. Die Anforderungen an einen sparsamen und nachhaltigen Umgang mit Wasser steigen stetig. Bereits heute wird vielerorts auf ressourcenschonende Systeme wie Tröpfchenbewässerung umgestellt. Eine intelligente Steuerung kann diesen Trend weiter unterstützen, indem sie den Wasserverbrauch anhand aktueller Sensordaten zusätzlich optimiert und nur dann bewässert, wenn es tatsächlich notwendig ist.

Die Inspiration stammt sowohl aus eigenen Erfahrungen im Umgang mit Technik und vom landwirtschaftlichen Familienbetrieb als auch aus dem Wunsch, ein praxisnahes Projekt mit direktem Nutzen für den Alltag umzusetzen.

## 1.3 Was bedeutet EIC

**EIC** steht für **Easy Irrigation Control**. Der Name beschreibt die grundlegende Funktion des Systems: eine einfache und effiziente Steuerung der Bewässerung. Ziel ist es, den Wasserverbrauch zu verringern und gleichzeitig eine benutzerfreundliche Lösung für Landwirte und Gartenbesitzer bereitzustellen.

## 1.4 Anwendungs-/Erweiterungsmöglichkeiten

Das System ist vielseitig einsetzbar. Neben der klassischen Bewässerung von Obstanlagen kann es auch in Gärten, Gewächshäusern oder für vertikale Landwirtschaftssysteme verwendet werden. Dank seines modularen Aufbaus lässt sich das System leicht erweitern – zum Beispiel durch zusätzliche Sensortypen (z. B. pH-Sensoren), neue Steuerungsmodi oder durch Integration weiterer Aktoren wie Pumpen oder Lüftungseinheiten. Auch die Softwarearchitektur bietet Potenzial für spätere Updates, etwa mit intelligenten Alarmfunktionen und einer detaillierten Auswertungs- und Statistikfunktion.

## 1.5 Abstract Italiano

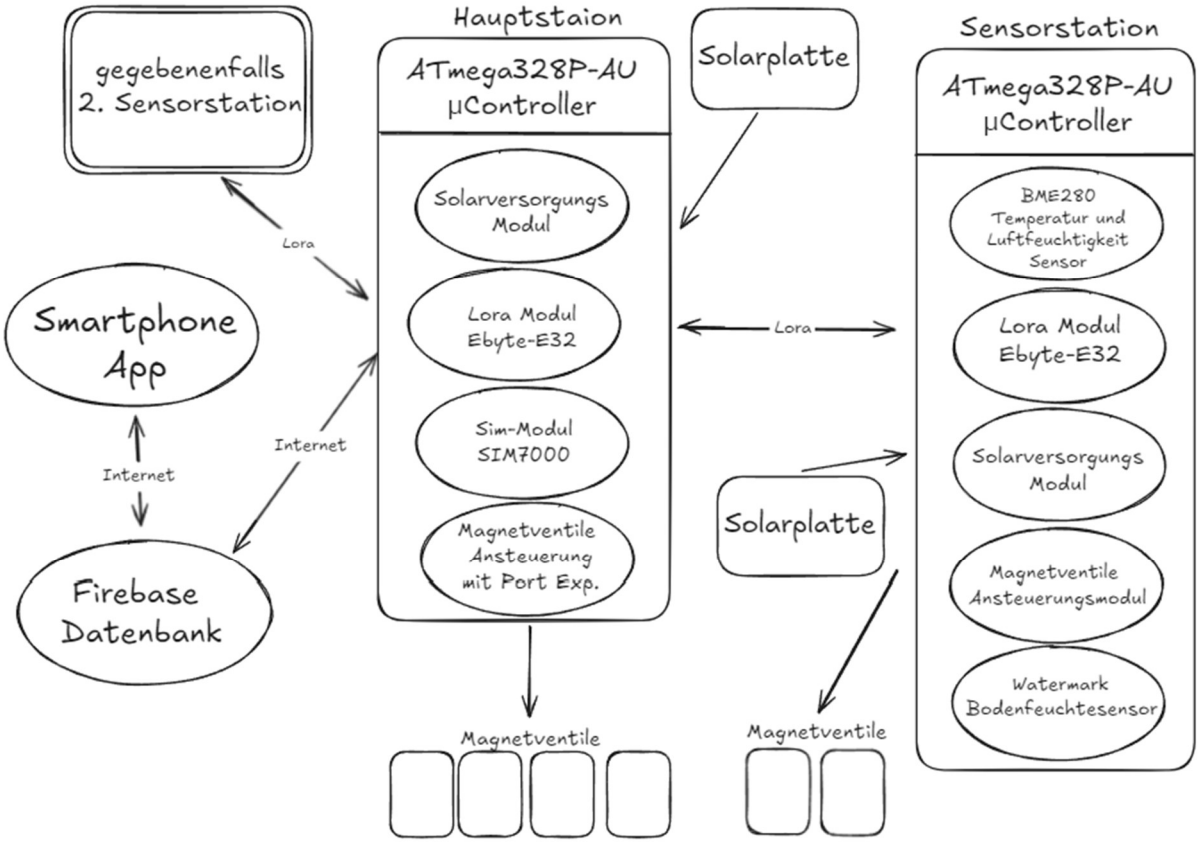
Il sistema EIC è una soluzione intelligente e pratica, pensata per rendere l'irrigazione dei campi e dei giardini più efficiente e sostenibile. Utilizza sensori per misurare l'umidità del terreno, così l'acqua viene usata solo quando è davvero necessaria. Questo è molto importante, perché l'acqua è sempre più limitata in agricoltura. Il sistema può accendere e spegnere l'irrigazione in modo automatico, in base all'umidità. Allo stesso tempo, è abbastanza semplice ed economico da poter essere usato anche da piccoli agricoltori o in giardini privati. L'utente può controllare tutto tramite un` app, che mostra le condizioni attuali e permette di cambiare facilmente le impostazioni. Il sistema è anche flessibile e può essere ampliato con altri sensori o valvole, se necessario. L'idea è nata dalla nostra fattoria di mele. Ho sempre voluto creare qualcosa di utile nella vita reale e, ricordando quanto spesso mio padre parlava dei vantaggi di un sistema di irrigazione intelligente, ho deciso che questo sarebbe stato il progetto perfetto.

## 1.6 Abstract English

The EIC system is a smart and practical solution designed to make watering fields and gardens more efficient and sustainable. By using sensors that measure how dry the soil is, it ensures water is only used when necessary. This is especially important as water is becoming more limited in agriculture. The system can turn the water on and off automatically, according to the sensor readings. At the same time, it is simple and affordable enough for small farms or home gardens, making smart irrigation accessible to more people. Users can control the system through a smartphone app, which shows current conditions and allows settings to be adjusted easily. The system is also flexible and can be expanded with additional sensors or valves if needed. The idea came from our own apple farm at home. I always wanted to create something with a real-world use case, and when I remembered how often my dad talked about the benefits of a smart irrigation system, I quickly decided this is the perfect project.

# 2 Gesamtkonzept

## 2.1 Blockschaltbild




Dieses Blockschaltbild schafft einen klaren Überblick über die wichtigsten Komponenten des Projekts und deren Zusammenspiel. Es ist so gestaltet, dass auf den ersten Blick ersichtlich wird aus welchen Elementen das Projekt besteht und wie diese zusammenarbeiten. Es zeigt auch anschaulich wie die Hauptstation und eine vollständig ausgestattete Sensorstation aufgebaut sind.

Auch die Verbindung zur Smartphone-App und zur Firebase-Datenbank wird anschaulich dargestellt, ebenso wie die Möglichkeit zur Integration weiterer Sensorstationen.

Die grafische Darstellung veranschaulicht, wie alle einzelnen Komponenten miteinander kommunizieren – sei es über LoRa, Internet oder über direkte Hardwareverbindungen. In den folgenden Kapiteln werden diese Elemente im Detail beschrieben.

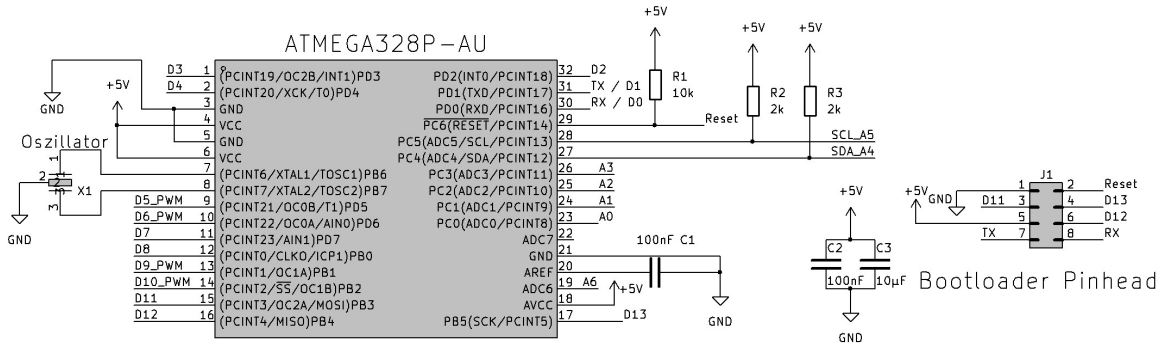
# 3 Hardware-Grundlagen

Beschreibung der wichtigsten Hardwarekomponenten.

## 3.01 Mikrocontroller

Im EIC-Bewässerungssystem wird sowohl auf der Sensorplatine als auch auf der Hauptplatine der Mikrocontroller **ATmega328P-AU** eingesetzt, der auch im Arduino Nano verwendet wird. Die Wahl fiel auf diesen Mikrocontroller, da bereits in vorherigen Projekten umfangreiche Erfahrungen mit ihm gesammelt wurden und somit eine vertraute Entwicklungsumgebung gegeben war. Ein weiterer entscheidender Faktor war die Anforderung, den Mikrocontroller standalone, also ohne Arduino-Board, direkt als IC in die Schaltung zu integrieren. Diese Anforderung konnte mit dem ATmega328P-AU problemlos umgesetzt werden.

Um den Mikrocontroller standalone zu betreiben, sind einige zusätzliche Bauteile nötig. Diese sind: ein 16 MHz Quarzoszillator, einige Kondensatoren und Widerstände sowie ein USB-TTL-Converter-System, um den Controller zu programmieren. Dieses habe ich allerdings auf die Programmiergerät-Platine ausgelagert. Außerdem werden zum Brennen des Bootloaders auch die Pins der SPI-Schnittstelle benötigt. Im untenstehenden Bild kann man die Grundschaltung des ATmega erkennen.

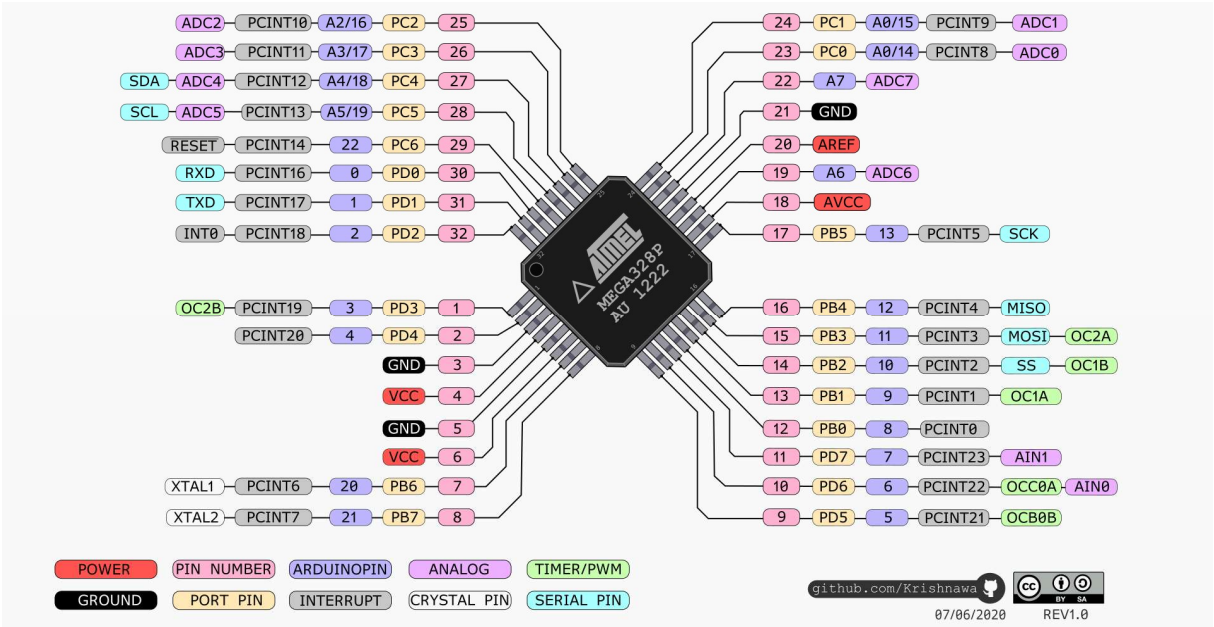



Neben der einfachen Integration zeichnet sich der Atmega328P-AU durch einen relativ geringen Stromverbrauch aus. Das ist von Vorteil, da das Projekt Batteriebetrieben ist. Da das EIC-Bewässerungssystem direkt im Feld eingesetzt wird, muss der Energieverbrauch so niedrig wie möglich gehalten werden, um die Betriebsdauer zu maximieren.

Der Mikrocontroller übernimmt im Gesamtsystem mehrere zentrale Aufgaben: Er steuert die Kommunikationsmodule SIM und LoRa, liest verschiedene Sensoren aus (z. B. Bodenfeuchtesensoren, Temperatursensoren) und berechnet darauf basierend relevante Werte für die Steuerung der Bewässerung.

Im Laufe der Entwicklung zeigte sich jedoch, dass die Rechenleistung des Atmega328P-AU an ihre Grenzen stößt. Aufgrund der Vielzahl an Modulen und Aufgaben kam es wiederholt zu Herausforderungen, alle Funktionen stabil und gleichzeitig zu realisieren. Diese Limitierung erforderte Optimierungen sowohl auf Hardware- als auch auf Softwareebene, um einen zuverlässigen Betrieb sicherzustellen.

Die Programmierung des Mikrocontrollers erfolgt mit C++ unter Verwendung der Entwicklungsumgebung PlatformIO in Visual Studio Code. Diese Kombination ermöglicht eine reibungslose Entwicklung sowie eine einfache Integration externer Bibliotheken.



## 3.02 Sim-Modul

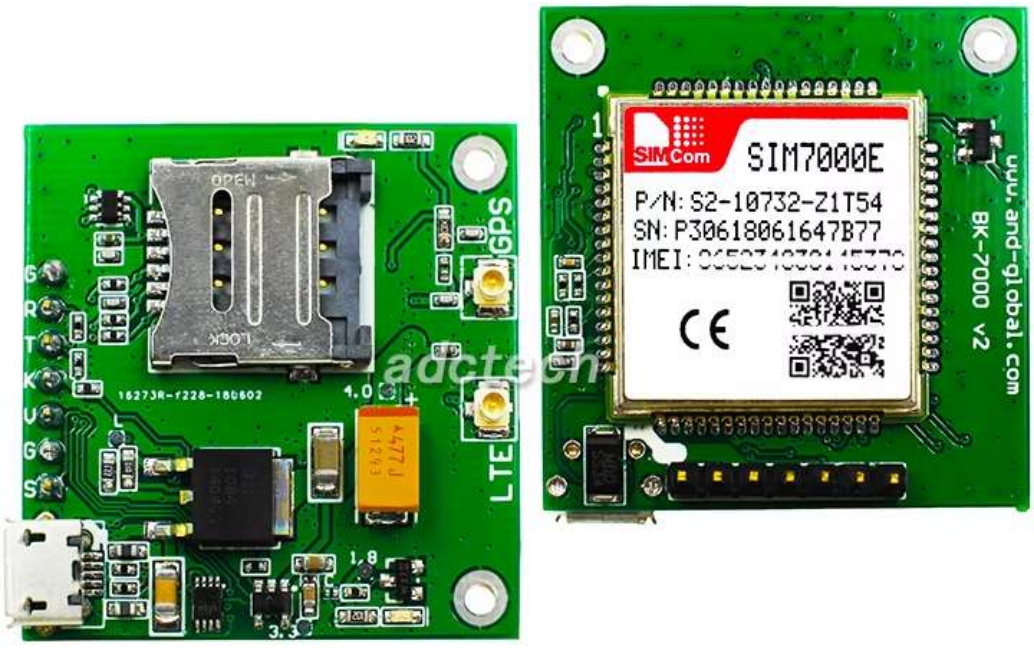
Für die Mobilfunkkommunikation des EIC-Bewässerungssystems wird ein **SIMCOM SIM7000**-Modul eingesetzt. Ursprünglich war der Einsatz eines **SIMCOM SIM7080**-Moduls geplant. Aufgrund technischer Schwierigkeiten bei der Inbetriebnahme wurde jedoch auf das SIM7000-Modul umgestiegen, welches sich als zuverlässiger erwies.

Im Gegensatz zum weitverbreiteten SIM800 welches nur bis zu 2G unterstützt kann das SIM7000 Modul mit bis zu 4G arbeiten. Weil das 2G System in den meisten Ländern (auch Italien) schon abgebaut wird, fiel meine Wahl auf das SIM7000 Modul.

Das SIM7000-Modul dient als zentrale Schnittstelle zwischen der Hauptplatine und der externen Infrastruktur. Es ermöglicht die Datenübertragung über das Mobilfunknetz und stellt die Verbindung zwischen dem EIC-System und der Datenbank (Firebase) sowie der zugehörigen Smartphone-App her.

Die Kommunikation mit dem Modul erfolgt über eine UART-Schnittstelle mithilfe von AT-Befehlen, die über eine entsprechende Bibliothek angesteuert werden. Als SIM-Karte kommt eine Hologram IoT-SIM zum Einsatz, welche für IoT-Anwendungen optimiert ist und weltweite Konnektivität ermöglicht.

Eine Herausforderung bestand darin, dass das SIM7000-Modul keine HTTPS-Verbindungen unterstützt. Und die Datenbank (Firebase) nur HTTPS-Verbindungen erlaubt, um dieses Problem zu lösen, wurde ein Proxy-Server zwischengeschaltet, der die HTTP-Anfragen empfängt und als HTTPS anfragen an Firebase weiterleitet. Die genaue Implementierung dieser Lösung wird im Softwareteil genauer beschrieben.



## 3.03 Lora-Modul

Zur drahtlosen Kommunikation zwischen der Sensorplatine und der Hauptplatine wird im EIC-Bewässerungssystem ein LoRa E32-Modul eingesetzt. Dieses Modul wurde aufgrund seiner guten Ergebnisse in Online-Vergleichstests sowie der einfachen Integration in Arduino-Projekte mit Hilfe von vorhandenen Bibliotheken ausgewählt.

Das LoRa E32-Modul arbeitet auf dem lizenzfreien 868 MHz Frequenzband, über das Kommunikationssystem LPWAN (Low Power Wide Area Network), das ermöglicht eine große Reichweite von mehreren Kilometern, wobei das Modul nur einen geringen Stromverbrauch von maximal 120mA im Sendezeitpunkt und ansonsten immer unter 25mA aufweist. Die maximale Sendeleistung im 868 MHz ISM-Band wird in Europa durch die ETSI-Norm *EN 300 220* geregelt. Diese Norm legt unter anderem die zulässige Ausgangsleistung und die Frequenzbereiche fest. Für das E32-Modul bedeutet dies eine maximal erlaubte Sendeleistung von +20 dBm (100 mW).

Die Ansteuerung des Moduls erfolgt über eine UART-Schnittstelle, was die Implementierung und Verwendung relativ einfach gestaltet hat. Das Modul wird mit 3.3V versorgt, die anderen Pins können aber auch direkt mit den 5V Ausgängen des ATmega betrieben werden. Das Modul verfügt über insgesamt 7 Pins, wobei 2 davon Versorgung (VCC, GND) und 2 weitere die beiden UART-Pins (RX, TX) sind. Der AUX-Pin signalisiert den aktuellen Betriebszustand des E32-Moduls. Wenn der Pin auf HIGH ist, bedeutet es, dass das Modul bereit ist aber keine Übertragung oder andere Operation stattfindet. Wenn der Pin auf LOW wechselt, bedeutet das, dass das Modul gerade beschäftigt ist. Die beiden letzten Pins sind M0 und M1 durch die der Betriebsmodus eingestellt werden kann. (Abbildung)

Im EIC-System wird das Modul für die bidirektionale Kommunikation zwischen den beiden Basisplatinen verwendet: Die Sensorplatine überträgt die erfassten Sensordaten an die Hauptplatine, während die Hauptplatine ihrerseits Steuerbefehle, insbesondere den Ventilschaltzustand, an die Sensorplatine sendet.




| Mode(0-3) | M1 | M0 | Mode introduction | Remark |
|---|---|---|---|---|
| Mode 0 Normal | 0 | 0 | UART and wireless channel is open, transparent transmission is on. | The receiver must works in mode 0 or mode 1 |
| Mode 1 Wake-up | 0 | 1 | UART and wireless channel is opened. The difference between normal mode and wake-up mode is it will add preamble code automatically before data packet transmission so that it can awaken the receiver works in mode 2. | The receiver can works in mode 0, mode 1 or mode 2. |
| Mode 2 Power-saving | 1 | 0 | UART is disabled. Wireless module works at WOR mode (wake on radio). It will open the UART and transmit data after receive the wireless data. | 1,the transmitter must works in mode 1<br>2,transmitting is not allowed in this mode |
| Mode 3 Sleep | 1 | 1 | Parameter setting. | |

## 3.04 BME280 Sensor




Auf der Sensorplatine wird ein BME280-Sensor eingesetzt, der die Temperatur und die Luftfeuchtigkeit misst. Diese Messgrößen sind entscheidend, um die Bewässerung optimal an die aktuellen Umweltbedingungen anzupassen und so eine effiziente Ressourcennutzung zu gewährleisten.

Der BME280 wird über eine I2C-Schnittstelle angesteuert und als fertiges Modul integriert. Die Programmierung erfolgt mithilfe einer entsprechenden Arduino-Bibliothek, die eine einfache Anbindung und Auswertung der Sensordaten ermöglicht.

Die Wahl fiel auf den BME280, da er kostengünstig, einfach zu integrieren und gleichzeitig sehr genau ist. Während der Entwicklung traten keine besonderen Herausforderungen bei der Inbetriebnahme oder Nutzung des Sensors auf.

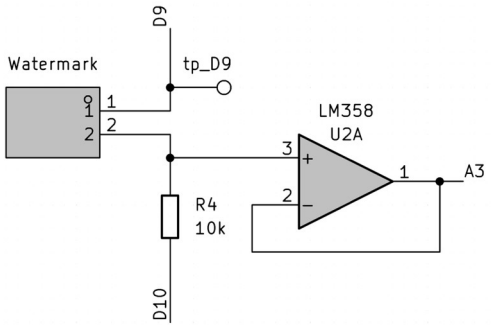
## 3.05 Watermark Sensor

Für die Messung der Bodenfeuchtigkeit wird im EIC-Bewässerungssystem ein Watermark Sensor vom Typ **Irrometer 200SS** eingesetzt. Im Gegensatz zu herkömmlichen kapazitiven oder resistiven Bodenfeuchtesensoren misst der Watermark-Sensor die **Wasserverfügbarkeit im Boden** indirekt über den elektrischen Widerstand innerhalb des Sensors, der sich abhängig vom für die Pflanzen verfügbaren Wasser verändert. Ein herkömmlicher Kapazitiver Bodenfeuchtesensor, kann im Gegensatz dazu nur die Feuchtigkeit des Bodens messen, was für das Bewässern der Pflanzen nicht sehr ausschlaggebend ist. Denn abhängig von der Bodenbeschaffenheit kann eine Pflanze auch bei hoher Feuchtigkeit nur wenig Wasser ziehen. Deshalb wird bei besseren Systemen, auch wenn der Kostenpunkt wesentlich höher ist, immer ein Saugspannungssensor verwendet.

Der Watermark-Sensor arbeitet resistiv und wird in die Messschaltung als Teil eines Spannungsteilers eingesetzt. Um den Widerstandswert des Sensors zu messen, wird für einen kurzen Zeitraum eine Spannung an den Spannungsteiler angelegt und die resultierende Spannung am Sensor abgelesen. Es ist wichtig, dass am Sensor nicht dauerhaft eine Spannung anliegt, da sonst Elektrolyseprozesse im Inneren die Messergebnisse verfälschen und die Lebensdauer des Sensors verkürzen würden.

Die Wahl fiel auf den Irrometer 200SS, da dieser Sensor für landwirtschaftliche Anwendungen optimiert, robust und verlässlich ist.

Die Auswertung des Sensors erfolgt über einen Impedanzwandler und einem analogen Eingang des Mikrocontrollers. Mithilfe des bekannten Widerstandswert im Spannungsteiler kann der Widerstand des Sensors berechnet und daraus auf die Bodenfeuchte geschlossen werden. Im Bild kann man die Schaltung erkennen, wobei die beiden Pins D9 und D10 für das zu und wegschalten der Versorgungsspannung genutzt werden.






## 3.06 H-Brücke und Magnetventil

Für die Ansteuerung der bistabilen Magnetventile im EIC-Bewässerungssystem wird der L298 Dual H-Bridge IC eingesetzt. Die verwendeten bistabilen Magnetventile benötigen lediglich einen kurzen Spannungsimpuls, um den Zustand zu wechseln, und halten diesen anschließend ohne weiteren Energieverbrauch. Diese Eigenschaft ist besonders vorteilhaft für den batteriebetriebenen Aspekt.

Die H-Brücke dient dazu, die Richtung des Stromflusses durch die Magnetventilspule zu steuern, sodass das Ventil umgeschaltet werden kann. Dafür müssen die H-Brücken eine Spannung von 12Volt schalten. Die Ansteuerung der H-Brücken erfolgt direkt über Digitalpins des Mikrocontrollers.

Auf der Hauptplatine werden zwei L298-ICs verwendet, um vier Magnetventile ansteuern zu können. Auf der Sensorplatine wird ein weiterer L298-IC auf einer aufsteckbaren Ventilerweiterungsplatine eingesetzt, welcher 2 Ventile steuern kann.

Die Wahl fiel auf den L298, da er eine einfache und bewährte Lösung zur Ansteuerung von Gleichstromlasten darstellt.

Wichtig sind bei solchen H-Brücken immer die Freilaufdioden, weil eine Induktive last (Magnetventil) beim schalten Spannungsspitzen erzeugt, welche durch die Freilaufdioden gestoppt werden.

Im Bild rechts ist ein Beispiel für ein solches Magnetventil dargestellt.



## 3.07 Port Expander

Auf der Hauptplatine wird ein MCP23017 Port Expander eingesetzt, um die Anzahl der verfügbaren GPIOs (Digital Pins) zu erweitern. Er wird benötigt, da der ATmega nicht über genügend Pins verfügt, um die vier Magnetventile über die H-Brücken und die ganzen anderen Bauteile direkt anzusteuern.

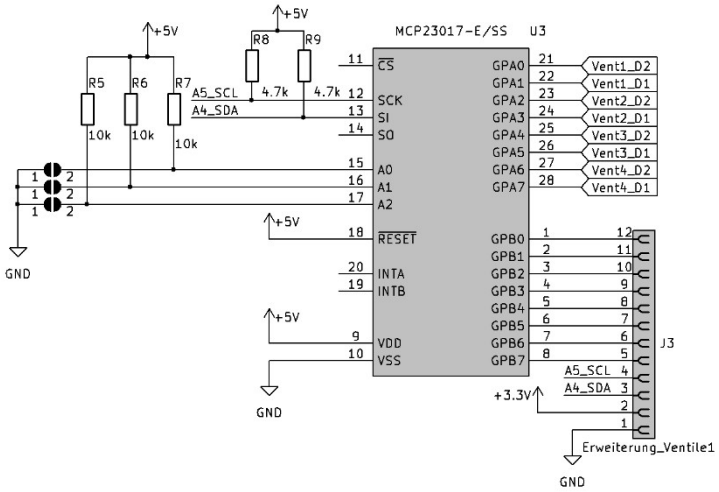
Der MCP23017 wird über die I2C-Schnittstelle an den Mikrocontroller angebunden und dient ausschließlich zur Ansteuerung der Steuersignale der H-Brücken (L298 ICs). So konnte die notwendige Anzahl an Steuerausgängen realisiert werden, ohne die Komplexität der Hauptplatine stark zu erhöhen.

Während der Entwicklung zeigte sich jedoch ein technisches Problem: Der gleichzeitige Betrieb des MCP23017 mit dem SIM-Modul führte zu Fehlfunktionen, die höchstwahrscheinlich auf Konflikte mit internen Timern oder der I2C-Implementierung zurückzuführen sind. Eine endgültige Lösung dieses Problems konnte im Rahmen des Projekts nicht vollständig umgesetzt werden und erfordert eine detaillierte Untersuchung der Library-Kompatibilität und Timer-Nutzung.

Ein weiteres Problem trat bei der Bauteilbestellung auf: Es gibt zwei nahezu baugleiche Modelle – den MCP23017 und den MCP23S17. Der entscheidende Unterschied besteht darin, dass der MCP23017 über I2C und der MCP23S17 über SPI angesteuert wird. Dies habe ich bei der Bestellung jedoch übersehen und versehentlich den MCP23S17 bestellt und aufgelötet.

Als das Bauteil im I2C-Scanner-Programm nicht erkannt wurde, begann ich mit der Fehlersuche. Schließlich stellte sich heraus, dass ich schlicht den falschen IC bestellt hatte. Nachdem ich den korrekten MCP23017 eingebaut hatte, funktionierte der Port Expander auf Anhieb problemlos.

Die Wahl des MCP23017 fiel aufgrund der einfachen Ansteuerung über I2C und der breiten Unterstützung durch bestehende Arduino-Bibliotheken.






## 3.08 Li-Ion Akku

Im EIC-Bewässerungssystem kommen Standard Lithium Ionen 18650 Akkuzellen für die Stromversorgung zum Einsatz. Diese haben eine Nominale Spannung von 3.7Volt und vollgeladen ca. 3000mAh. Sie wurden gewählt, da sie kostengünstig, leicht verfügbar und für die erforderlichen Kapazitäts- und Spannungsanforderungen geeignet sind. Die Hauptplatine und die Sensorplatine werden jeweils mit 2 18650 Akkus versorgt, welche auf der Platinen Rückseite mittels einer speziellen Halterung angebracht sind. Ohne aufladen habe ich dadurch eine Akkulaufzeit von ca. einer Woche erreicht.

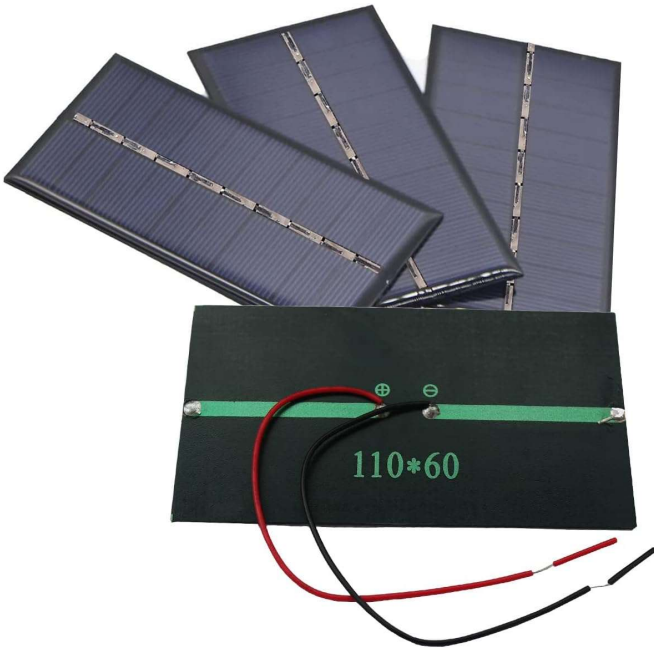
Die Akkus werden über einen TP5400 Laderegler geladen, der über die angeschlossene Solarzelle mit Energie versorgt wird. Dieses Bauteil und auch die gesamte Solarversorgunsplatine werden im laufe der Dokumentation noch genauer beschrieben.




Die Lithium-Ionen-Akkus ermöglichen eine kompakte, wiederaufladbare und netzunabhängige Energieversorgung, die insbesondere für den Einsatz im Feld wo meist kein Netzstrom vorhanden ist, sehr wichtig ist.

## 3.09 Solarzelle

Zum Aufladen der 18650 Lithium Akkus wird eine Standardsolarzelle (110 x 60mm) mit 6 V Nennspannung und einer maximalen Stromstärke von 150 mA verwendet, welche die Akkus über den TP5400 auflädt. Alternativ zur Solarzelle ist es auch möglich ein normales 5Volt Netzteil zu verwenden, wobei der Ladestrom automatisch auf 500mA begrenzt wird. Eine diese Zellen reicht normalerweise aus um das System durchgehend am Laufen zu halten, falls diese jedoch nicht ausreicht, könnte man immer noch eine zweite Solarzelle Parallel dazu einbauen.



## 3.10 Durchflusssensor

Zusätzlich zu den Magnetventilen wird immer auch noch ein einfacher analoger Durchflusssensor verbaut. Der Sensor verfügt über drei Anschlüsse: +5 V, GND und ein analoges Ausgangssignal (0–5 V). Im Inneren arbeitet er mit einem rotierenden Flügelrad (Impeller), dessen Drehgeschwindigkeit proportional zur Durchflussmenge ist.

Die Hauptfunktion des Sensors besteht darin, zu überprüfen, ob das Magnetventil tatsächlich geöffnet ist und Wasser durch die Leitung fließt. Dadurch kann sichergestellt werden, dass ein Steuerbefehl auch zu einer tatsächlichen Wasserabgabe führt, und bei einer Fehlfunktion direkt ein Alarm über die App gemeldet werden.



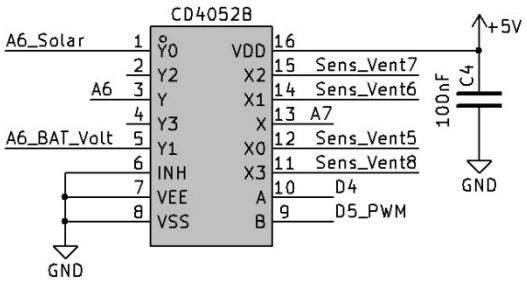
Der Sensor ist direkt mit einem analogen Eingang des Mikrocontrollers verbunden und wurde aufgrund seiner einfachen Handhabung, geringen Kosten und ausreichenden Genauigkeit für diesen Anwendungszweck ausgewählt.

## 3.11 Analogmultiplexer

Der **CD4052** ist ein analoger Multiplexer bzw. Demultiplexer, der zur Erweiterung der analogen Eingänge des Mikrocontrollers eingesetzt wird. Er ermöglicht es, mehrere analoge Signale (z. B. von Sensoren) über nur einen oder zwei ADC-Pins des Mikrocontrollers einzulesen.

Der CD4052 enthält zwei unabhängige 4:1 Multiplexer, was bedeutet, dass je vier analoge Eingangssignale auf jeweils einen Analog-Pin geschaltet werden können. Diese beiden Multiplexer werden parallel über dieselben beiden Adressleitungen gesteuert, mit welchen man Binär eines der 4 Eingangssignale auswählen kann.

Auf der Hauptplatine werden insgesamt 6 der 8 Möglichen Eingänge der beiden Multiplexer verwendet. Vier analoge Sensorsignale werden auf die Eingänge X0 bis X3 angeschlossen. Der durch die Adressleitungen A und B gewählte Eingang wird dann auf den Ausgang X durchgeschaltet, welcher direkt mit dem Analog-Pin des ATmega verbunden ist. Dasselbe passiert bei der Y Seite mit 2 Analogen Signalen auf Y0 und Y1.



## 3.12 DCDC-Wandler

Ein DC-DC-Wandler ist eine elektronische Schaltung, die eine Gleichspannung in eine andere Gleichspannung umwandelt. Dabei kann die Ausgangsspannung entweder niedriger (Step-Down, auch „Buck"-Wandler genannt) oder höher (Step-Up, auch „Boost"-Wandler genannt) sein als die Eingangsspannung.

DC-DC-Wandler kommen überall dort zum Einsatz, wo unterschiedliche Betriebsspannungen benötigt werden, aber nur eine Spannungsquelle vorhanden ist – zum Beispiel in batteriebetriebenen Geräten wie dem EIC-System. Der Vorteil dieser Wandler liegt in ihrer hohen Energieeffizienz gegenüber linearen Spannungsreglern, da sie die überschüssige Spannung nicht in Wärme abgeben, sondern sie durch getaktete Energiespeicherung und Freisetzung regeln. Dies ermöglicht einen Wirkungsgrad von bis zu 95%.

Die Funktion eines DC-DC-Wandlers basiert auf der Zwischenspeicherung von Energie in Spulen. Durch das schnelle Ein- und Ausschalten eines Schalters (z. B. MOSFET) wird eine geregelte Spannung erzeugt. Die Schaltfrequenz liegt typischerweise im Bereich von einigen hundert Kilohertz bis hin zu mehreren Megahertz. Ein Regler-IC überwacht dabei permanent die Ausgangsspannung und passt den Duty-Cycle der Schaltfrequenz an, um eine stabile Spannung zu gewährleisten.

In meinem Projekt wurden DC-DC-Wandler eingesetzt, um aus der 3.7V Batteriespannung 3.3V 5V und 12V für die Mikrocontroller und Sensoren zu erzeugen. Durch die hohe Effizienz der Wandler wird der Energieverbrauch reduziert und gleichzeitig die Wärmeentwicklung gering gehalten – ein wichtiger Faktor für batteriebetriebene Systeme.

## 3.13 TP5400 Laderegler

Der TP5400 ist ein kompakter Power-Management-IC, der speziell für das Laden von Einzelzellen-Lithium-Ionen-Akkus entwickelt wurde. Zusätzlich zur Ladefunktion stellt der TP5400 eine 5V-Ausgangsspannung bereit, sodass er sich ideal für mobile Geräte oder Projekte mit Akkuversorgung eignet.

Der Chip integriert einen Laderegler, einen Boost-Konverter und eine Schutzschaltung, wodurch kein weiteres externes Lade-IC benötigt wird.

Im EIC-System kommt der TP5400 auf der Soalrversorgunsplatine zum Einsatz, wo er die Energieversorgung der Sensoren übernimmt. Er lädt den eingebauten Li-Ion-Akku und stellt gleichzeitig über seinen 5 V-Ausgang die nötige Betriebsspannung für Mikrocontroller und Sensorik bereit. Diese Integration spart Bauraum und Komponenten, was das Design kompakter und effizienter macht.

Ein weiterer Vorteil ist der integrierte Schutz gegen Überladung, Tiefentladung und Kurzschluss, was die Betriebssicherheit erhöht. So trägt der TP5400 wesentlich zur Stabilität und Mobilität der Sensorplatine bei.

# 4 Software-Grundlagen

## 4.1 I2C

I²C (Inter-Integrated Circuit) ist ein serielles Protokoll zur Datenübertragung zwischen Mikrocontrollern und Peripheriegeräten wie Sensoren oder Displays. Es benötigt nur zwei Leitungen: **SDA** für Daten und **SCL** für das Taktsignal. Alle Geräte teilen sich diese Busleitungen, jedes mit einer eindeutigen Adresse. Der Mikrocontroller (Master) steuert die Kommunikation, und kommuniziert mit den Slaves.

Je nach Modus erreicht I²C Datenraten von 100 kHz bis 3,4 MHz und ist ideal für kurze Distanzen also direkt auf Platinen gut geeignet. Ein großer Vorteil ist die einfache Erweiterbarkeit: Mehrere Geräte können über denselben Bus angesprochen werden, was Pins und Verdrahtung spart.

Im EIC-System wird I²C für den **BME280** Sensor und den **Portexpander MCP23017** verwendet. Durch bestehende Bibliotheken ist die Anbindung einfach, was eine flexible, kompakte und kostengünstige Systemstruktur ermöglicht.

## 4.2 Bootloader

Ein Bootloader ist ein kleines Softwareprogramm, das beim Einschalten des Mikrocontrollers als erstes ausgeführt wird. Seine Hauptaufgabe besteht darin, das System zu initialisieren und anschließend die eigentliche Firmware zu laden und auszuführen. Der Bootloader sorgt auch dafür, dass z.B. im Fall vom Arduino der Mikrocontroller über UART programmiert werden kann. Er ist in der Regel im geschützten Speicherbereich abgelegt, um versehentliche Überschreibungen zu vermeiden. Für den ATmega328P wird der Standard-Arduino-Bootloader verwendet.

Der Bootloader wird normalerweise über die SPI-Schnittstelle (MOSI, MISO, SCK) hochgeladen, indem diese mit gewissen Pins eines bereits vorhandenen Arduino Nano verbunden werden. Dabei muss man außerdem noch einige weitere Pins wie RESET und die Versorgungsspannung verbinden. Um diesen Prozess zu erleichtern, habe ich ein Programmiergerät entwickelt, das durch Einstecken über einen 2×4-Pinhead den Bootloader auf den Chip brennt. Anschließend kann es dann auch für das normale Hochladen des Codes verwendet werden.
Eine detaillierte Beschreibung dieses Geräts gibt es in einem späteren Abschnitt.

## 4.3 Datenbank Firebase

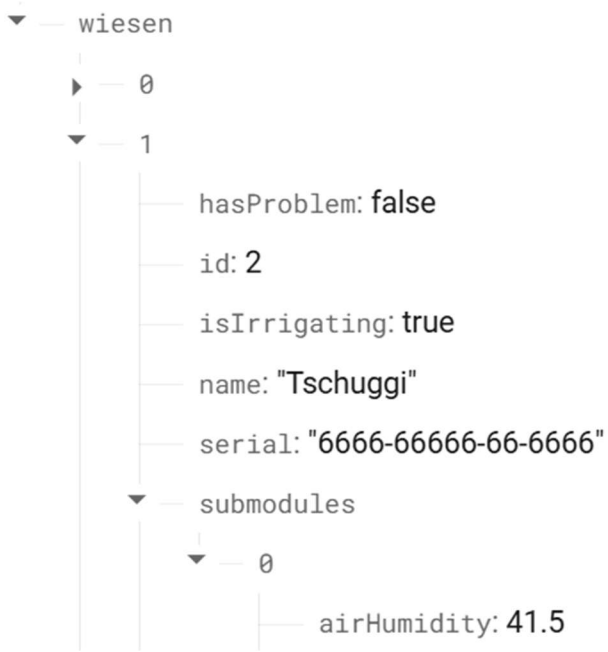
Im EIC-Bewässerungssystem wird Firebase als zentrales Backend eingesetzt. Firebase übernimmt dabei zwei Hauptaufgaben: Zum einen die Authentifizierung der Benutzer innerhalb der mobilen App, zum anderen die Speicherung und Bereitstellung aller wichtigen daten in der Firebase Realtime Database.

Sowohl die Hauptplatine (über das SIM-Modul) als auch die mobile App greifen direkt auf diese Datenbank zu, um Daten auszutauschen und das System zu steuern.

Die Entscheidung für Firebase fiel aufgrund der All-in-One-Lösung, die Benutzerauthentifizierung, Datenbank, Hosting und weitere Backend-Dienste nahtlos integriert. Damit konnte der Entwicklungsaufwand reduziert und eine zuverlässige Cloud-Lösung ohne eigenen Server realisiert werden.

Eine besondere Herausforderung stellte die Tatsache dar, dass Firebase ausschließlich über HTTPS kommuniziert, das verwendete SIM7000-Modul jedoch kein HTTPS unterstützt. Aus diesem Grund wurde ein Proxy-Server zwischengeschaltet, der die HTTPS-Anfragen entgegennimmt und in unverschlüsselte HTTP-Anfragen für das SIM-Modul umwandelt. Auf die technische Umsetzung dieser Lösung wird im Softwareteil noch detaillierter eingegangen.

Im untenstehenden Bild sieht man einen kurzen Ausschnitt aus der Datenbank. Die Daten darin werden im JSON-Format gespeichert.

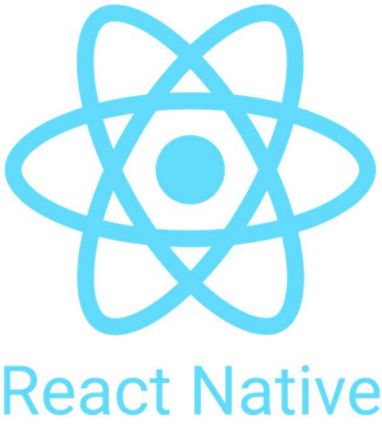


## 4.4 React Native

React Native ist ein plattformübergreifendes Open-Source-Framework, das von Meta (ehemals Facebook) entwickelt wurde. Es ermöglicht die Entwicklung nativer mobiler Anwendungen für iOS und Android unter Verwendung von JavaScript und der React-Bibliothek. Anstatt die App vollständig in den nativen Programmiersprachen (z. B. Swift oder Kotlin) für das jeweilige Betriebssystem zu schreiben, erlaubt React Native die Wiederverwendung einer gemeinsamen Codebasis, die zu nativen Komponenten kompiliert wird. Dies bietet eine nahezu native Benutzererfahrung bei deutlich reduziertem Entwicklungsaufwand.

Die Plattformunabhängigkeit ist deshalb auch der größte Vorteil von React Native, durch die gemeinsame Codebasis lassen sich Anwendungen gleichzeitig für Android und iOS entwickeln, was den Entwicklungs- und Wartungsaufwand erheblich reduziert. Darüber hinaus unterstützt das Framework eine schnelle und effiziente Entwicklung durch eine große Auswahl an verfügbaren Bibliotheken und der Expo Go App. Diese ermöglicht es, Anwendungen ohne vorherige Kompilierung direkt auf einem realen Mobilgerät zu testen und Änderungen in Echtzeit zu aktualisieren. In vielen Anwendungsfällen ermöglicht React Native dank der Verwendung nativer UI-Komponenten eine Performance, die der von vollständig nativen Apps sehr nahekommt.

Trotz dieser Vorteile gibt es auch Einschränkungen. Bei komplexen, hardware- oder betriebssystemnahen Funktionen kann es notwendig sein, zusätzliche native Module zu integrieren, was die Komplexität der Entwicklung erhöht. Zudem stößt React Native bei besonders rechenintensiven oder Grafik lastigen Anwendungen, wie etwa Spielen, an Performancegrenzen. Allerdings sind das keine Nennenswerte Einschränkungen, weshalb die Wahl für meine App, relativ schnell auf React Native gefallen ist.



# 5 Hardware

Das gesamte System wurde bewusst modular aufgebaut, indem die Funktionen auf mehrere Unterplatinen verteilt wurden. Diese Aufteilung bietet entscheidende Vorteile in Bezug auf Wartung, Erweiterbarkeit und Fehlersuche. Durch die Trennung der Hauptfunktionen – wie Stromversorgung, Sensorik, Steuerung und Aktorik – auf eigene Platinen kann jede Komponente unabhängig getestet und bei Bedarf ausgetauscht oder erweitert werden, ohne das Gesamtsystem zu beeinträchtigen.

Ein zentraler Gedanke hinter dem Konzept ist es, die Modularität sowohl auf Hardware- als auch auf Softwareebene zu ermöglichen. Alle Erweiterungsplatinen, wie etwa die Magnetventilsteuerung, sind über einheitliche Steckverbindungen mit den Basisplatinen verbunden. Die Pinheads sind eindeutig beschriftet und durch Markierungen gegen verdrehtes einstecken geschützt, um einen problemlosen Einbau zu gewährleisten.

Die Platinen wurden alle mit dem Open Source Programm KiCad gezeichnet. Und bei JLCPCB bestellt.

## 5.1 Hauptplatine (V1/V2)

### 5.1.1 Schaltungsbeschreibung

Die Hauptplatine bildet das Herzstück der EIC und vereint zahlreiche zuvor beschriebene Hardwarekomponenten auf einer kompakten Leiterplatte. Der zentrale Mikrocontroller ATmega328P-AU (Arduino-kompatibel) steuert alle angeschlossenen Module und übernimmt die gesamte Systemlogik.

Die Kommunikation mit den Sensorplatinen erfolgt über das LoRa E32 Modul, während der Datenaustausch mit der Online-Datenbank über das extern aufgesteckte SIM7080G-Modul abgewickelt wird. Für die Ansteuerung der vier Magnetventile kommt ein Portexpander (MCP23017) zum Einsatz, da die GPIOs des Mikrocontrollers allein nicht ausreichen würden.

Auch ein Steckplatz für ein DS3231 RTC Modul (I2C) wurde vorgesehen, welches sich allerdings als unnötig herausstellte, da die Zeit auch vom Sim Modul gelesen werden konnte.

Auf der Rückseite der Platine befindet sich eine Akkuhalterung für zwei 18650 Li-Ion-Zellen, welche die Stromversorgung der Hauptplatine übernehmen. Die Akkus werden über ein separates externes Solarlade-Modul geladen, das als eigene, austauschbare Platine realisiert wurde. Dies erhöht die Modularität des Systems erheblich und hat sich insbesondere bei der Fehlersuche und bei Weiterentwicklungen als großer Vorteil erwiesen.

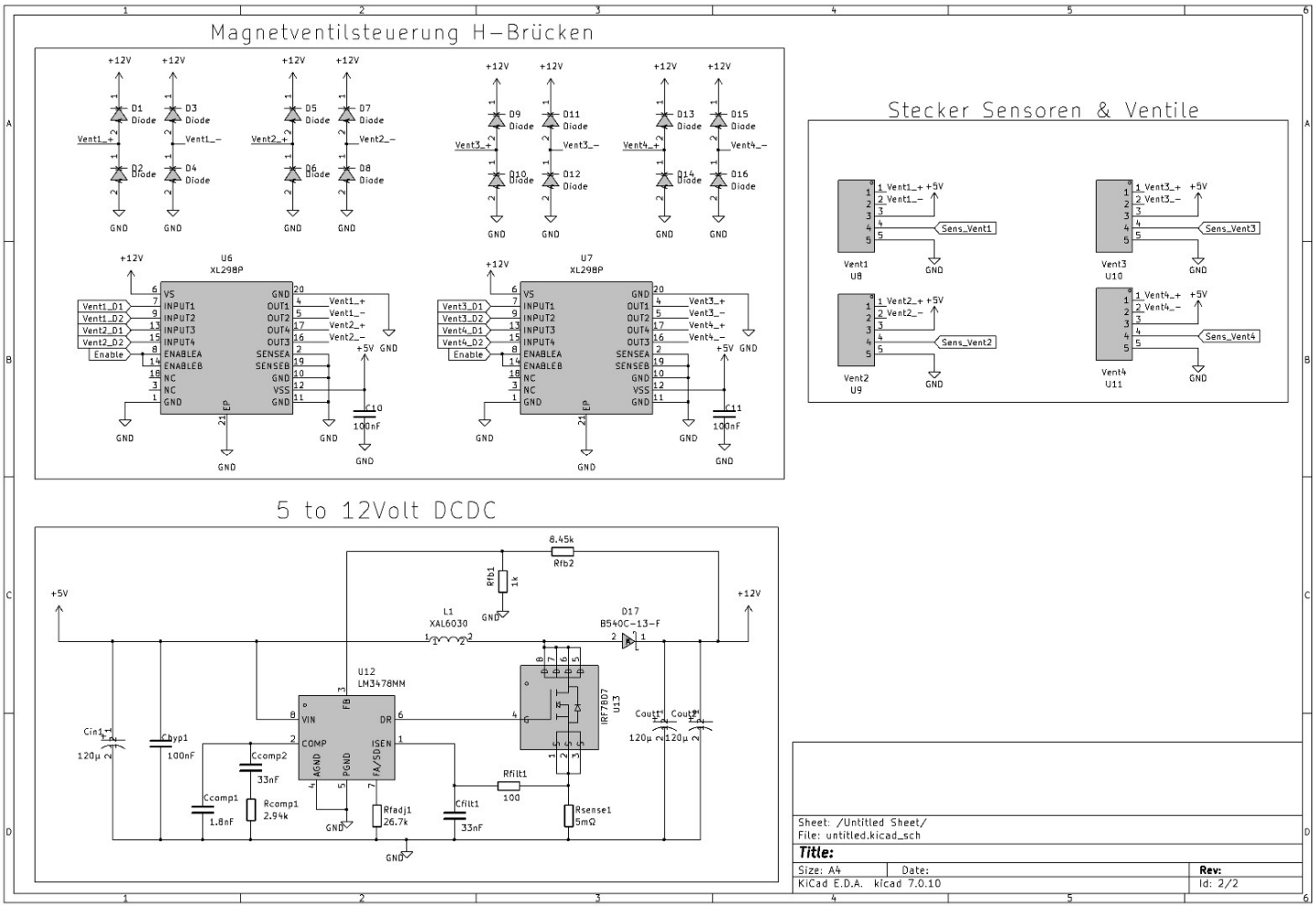
Da in manchen Anwendungsfällen vier Magnetventile nicht ausreichen könnten, wurde bereits bei der Entwicklung eine Erweiterungsmöglichkeit vorgesehen. Über

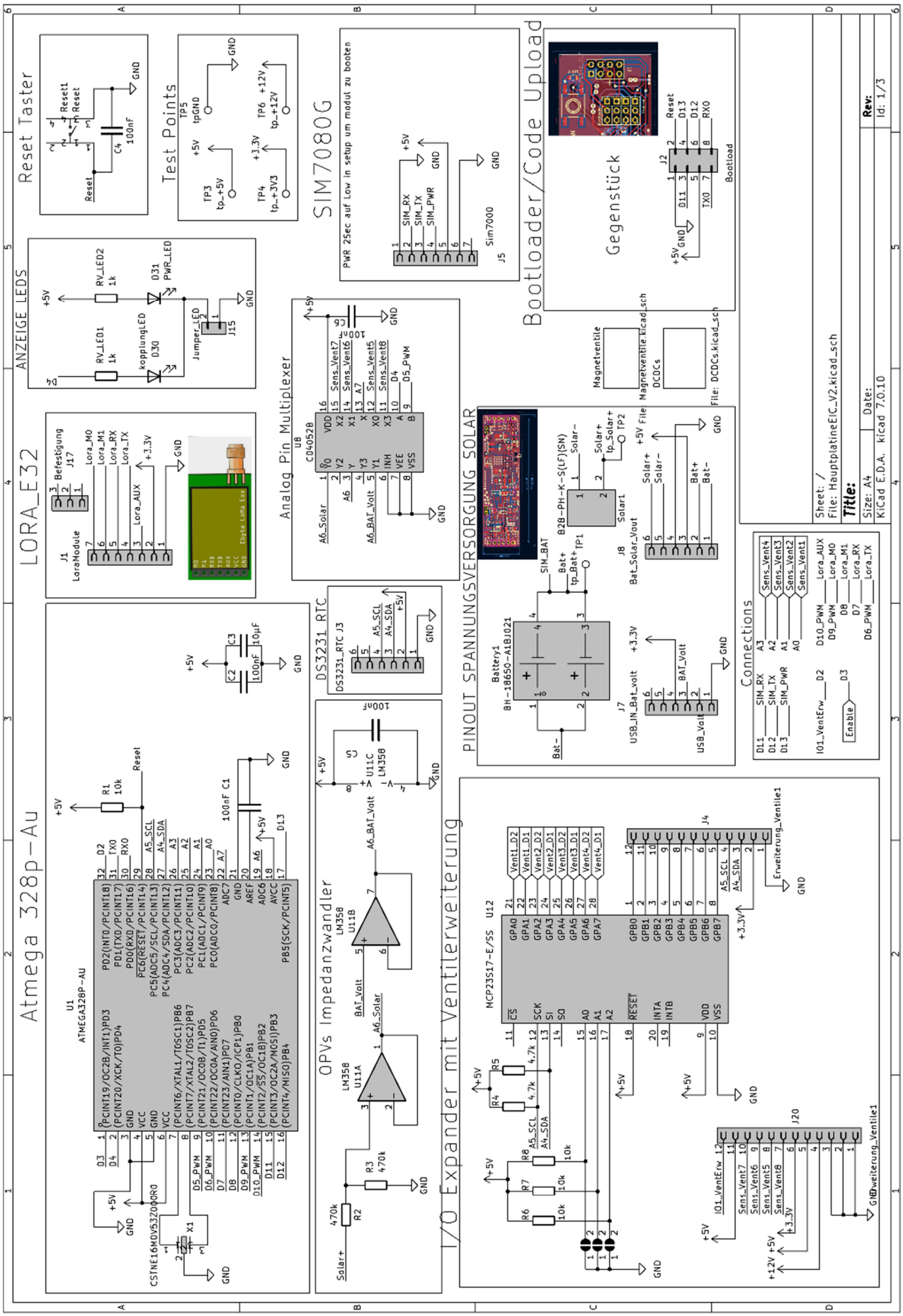
einfache Pinheader kann ein weiteres Ventilerweiterungsmodul direkt aufgesteckt werden.

Da mit der Möglichen Erweiterung bis zu 8 Durchflusssensoren und einige weitere Analogwerte ausgelesen werden müssen, wurde ein analoger Multiplexer eingebaut. Die Spannungen von Solarpanel und Akku werden über zwei separate ADC-Pins eingelesen und zuvor mithilfe eines Impedanzwandlers (LM358 OPV) aufbereitet, um die Eingänge zu schützen und ein sauberes Signal zu gewährleisten.

Für die Programmierung des Moduls wird ein 8-Pin Pinhead verwendet, durch den das externe Programmiergerät verbunden werden kann, dies verringert die Komplexität auf einer einzelnen Platine und dadurch auch die Fehlerquellen weiter, und bietet auch noch einige andere Vorteile, die im Abschnitt zum Programmiergerät genauer erläutert werden.

In den nachfolgenden Bildern ist die komplette Schaltung der Hauptplatine V2 dargestellt.




Reset Taster
Test Points
SIM7080G
PWR 2Sec auf Low in setup um modul zu booten
ANZEIGE LEDS
Bootloader/Code Upload
Gegenstück
LORA_E32
Analog Pin Multiplexer
DS3231 RTC
PINOUT SPANNUNGSVERSORGUNG SOLAR
Atmega 328p-Au
OPVs Impedanzwandler
I/O Expander mit Ventilerweiterung
Connections
Sheet: /
File: HauptplatineEIC_V2.kicad_sch
Title:
Size: A4
Date:
KiCad E.D.A. kicad 7.0.10
Rev:
Id: 1/3

### 5.1.2 Platinen Designconsiderations

Ein zentraler Aspekt beim Layout der Platine war die Einhaltung eines kompakten Formfaktors (80 x 100mm), um die Platine möglichst platzsparend in ein Gehäuse integrieren zu können und so die Installation auch in beengten Umgebungen zu ermöglichen und den Preis der Platine niedrig zu halten. Gleichzeitig wurde großer Wert auf Übersichtlichkeit und Servicefreundlichkeit gelegt – ein bei Prototypen wichtiger Punkt, um später Änderungen, Reparaturen oder Erweiterungen leichter durchführen zu können.

Aufgrund der hohen Schaltungskomplexität und auch dank eines günstigen Angebots des Herstellers JLCPCB fiel die Entscheidung zugunsten einer sechs-Layer Platine. Dies ermöglichte die Platzierung separater Versorgungspolygone für +5 V, +3.3 V und GND auf den inneren Layern. Dadurch blieben sowohl der oberste (Top) als auch der unterste (Bottom) Layer sowie Layer 4 weitgehend frei von Versorgungsleitungen, was erheblich Platz für die Signalführung schuf und dar Routen erleichterte.

Ein weiterer wichtiger Aspekt war die Integration von Testpunkten. Diese ermöglichen es, während der Inbetriebnahme und Fehlersuche schnell und zuverlässig wichtige Versorgungsspannungen und Signale zu überprüfen.

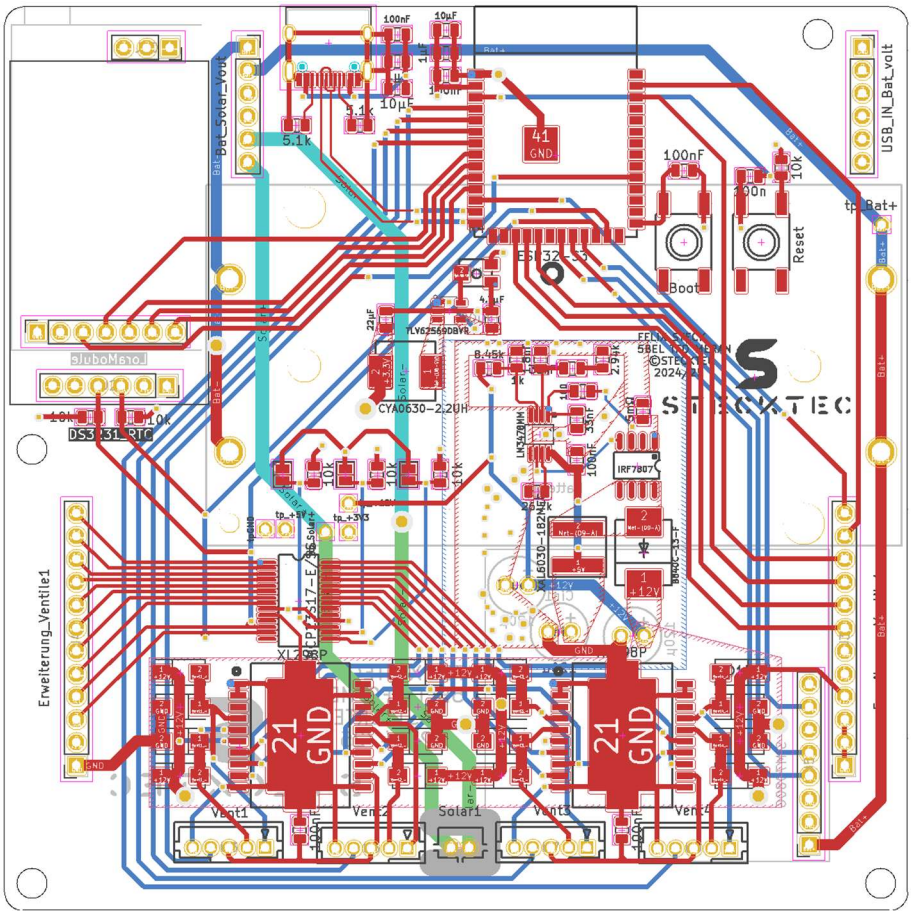
Um zusätzlichen Platz auf der Vorderseite zu gewinnen, wurden einige größere Bauteile wie das SIM-Modul, Elektrolytkondensatoren und der 18650-Batteriehalter auf die Rückseite der Platine verlagert.

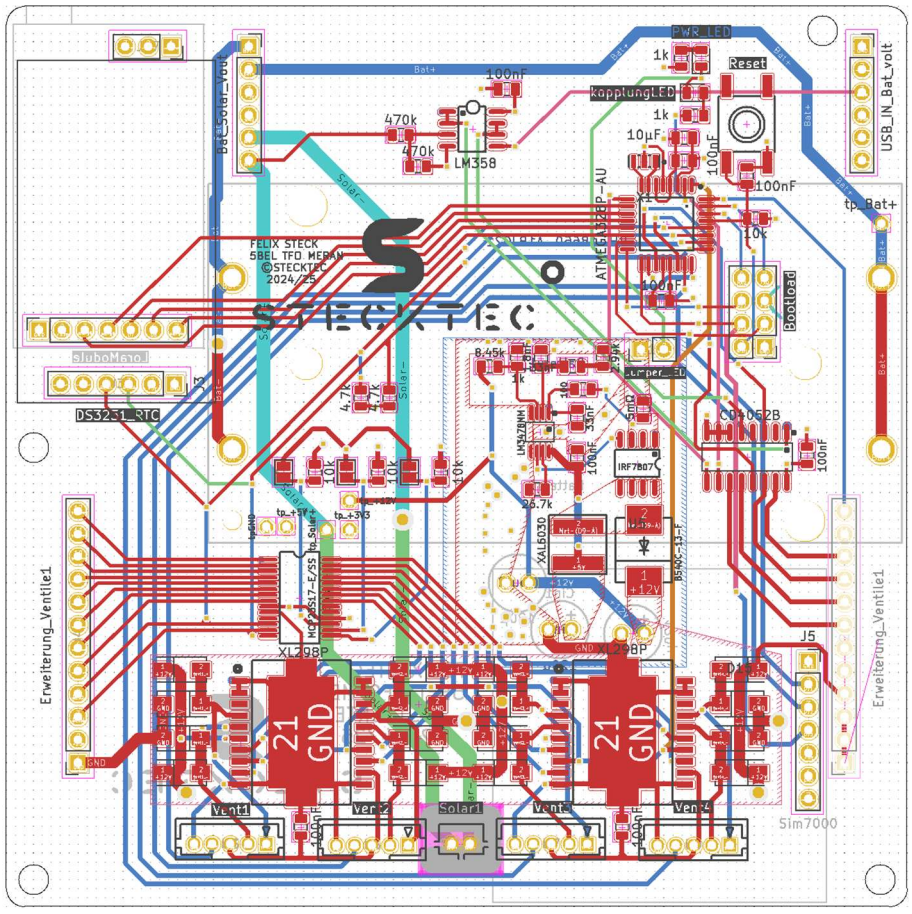
Sämtliche Bauteile wurden bewusst so ausgewählt, dass eine komplette Bestückung von Hand möglich ist, um bei Bedarf auch einzelne Komponenten einfach austauschen oder nachlöten zu können – ein praktischer Vorteil insbesondere beim Prototypenbau.

Zum einfachen Testen und Debuggen des Mikrocontrollers wurde eine Status-LED an Pin D4 integriert. Zusätzlich sorgt eine Power-LED dafür, dass der Betriebszustand der Platine jederzeit optisch erkennbar ist.

Da die Magnetventile mit 12 V betrieben werden müssen, kommt ein Step-Up-Wandler (5 V auf 12 V) (LM3478) zum Einsatz. Derselbe DCDC wird auch auf der Sensor Ventilerweiterungsplatine verwendet. Die Dimensionierung des DCDC-Wandlers erfolgte mit dem Tool WEBENCH® von Texas Instruments, welches neben der Auswahl der Bauteile auch ein optimiertes Layout mit konkreten Platzierungs- und Routingvorschlägen lieferte. Dadurch konnte eine hohe Effizienz und ein zuverlässiger Betrieb des DCDCs gewährleistet werden.

**Im Bild oben Platine Version 1 und unten Version 2**



### 5.1.3 Probleme mit V1

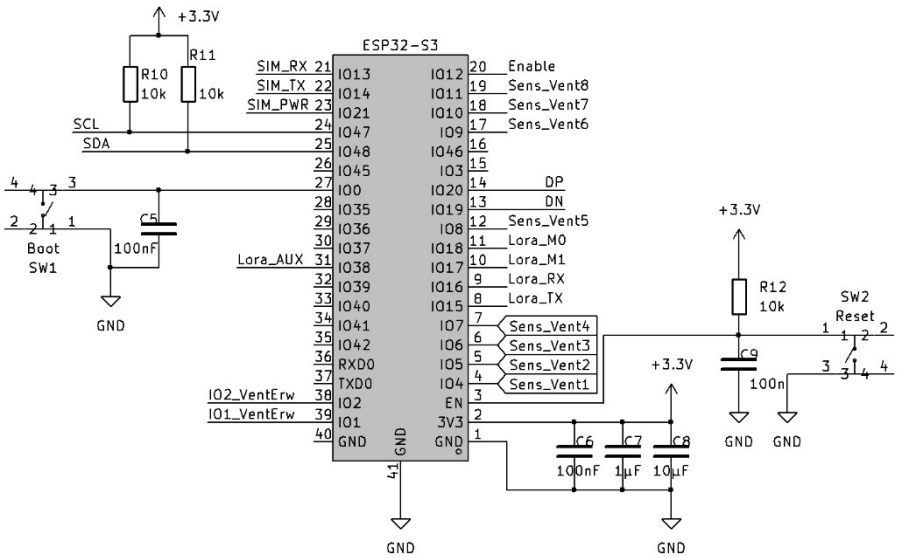
Da die erste Version der Hauptplatine mehrere entscheidende Schwächen aufwies, wurde eine überarbeitete zweite Version notwendig. Ursprünglich war vorgesehen, einen ESP32-S3 Mikrocontroller zu verwenden. Dieser sollte durch seinen integrierten USB-TTL-Wandler eine einfachere Programmierung ermöglichen als der klassische Arduino. In der Praxis zeigte sich jedoch, dass das Hochladen von Programmen extrem unzuverlässig war – oft funktionierte der Upload erst nach mehreren Versuchen, und die genaue Ursache ließ sich nicht eindeutig feststellen.

Ein weiteres wesentliches Problem war der Stromverbrauch des ESP32-S3. Trotz deaktivierter Bluetooth- und WLAN-Funktionalität lag der Idle-Stromverbrauch bei über 100 mA, was für ein batteriebetriebenes System mit kleinem Solarpanel viel zu hoch ist und meine Erwartungen deutlich überstieg.

Parallel dazu arbeitete ich während der Wartezeit auf die ersten Platinen intensiv an der Inbetriebnahme des SIM7080G-Moduls, das sich jedoch als nicht funktional erwies. Erst der Umstieg auf das SIM7000-Modul, das sich schließlich als zuverlässig erwies, brachte den gewünschten Erfolg. Diese Änderung machte ohnehin eine Modifikation der Platine notwendig.

Aufgrund dieser Probleme entschied ich mich dazu, die ESP32-Problematik nicht weiter zu verfolgen. Stattdessen entwickelte ich zügig eine neue Version V2 auf Basis des ATmega328P, den ich bereits erfolgreich auf der Sensorplatine im Einsatz hatte. Dies ermöglichte nicht nur eine stabile und energieeffiziente Lösung, sondern half auch, den Zeitplan einzuhalten.

Im Bild ist die Schaltung mit dem ESP32-S3 aus der 1.Version der Hauptplatine dargestellt, welcher dann in der V2 mit der Schaltung des ATmega ersetzt wurde.



## 5.2 Sensorplatine

Die Sensorplatine wird – genau wie die Hauptplatine – von einem Arduino-Mikrocontroller gesteuert. Dabei kommt dasselbe externe Programmiergerät zum Einsatz, was die Komplexität reduziert, und die Entwicklung vereinfacht.

Für die Kommunikation mit der Hauptplatine wird erneut das LoRa E32-Modul verwendet, wodurch eine stabile Funkverbindung zwischen den beiden Einheiten gewährleistet ist. Zusätzlich wurde der BME280-Sensor zur Messung von Temperatur und Luftfeuchtigkeit über die I²C-Schnittstelle integriert.

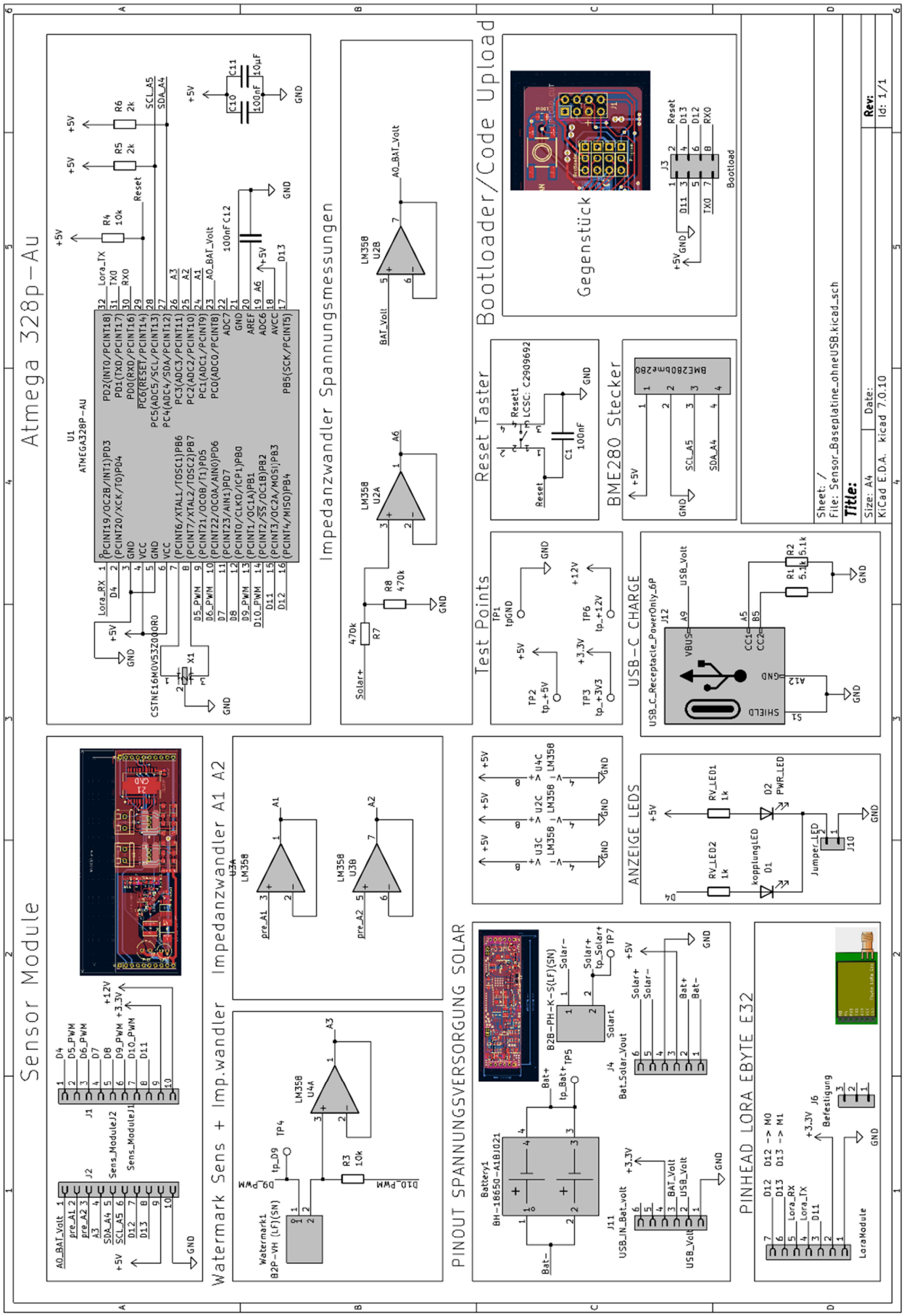
Auch die Messschaltung für den Watermark-Bodenfeuchtigkeitssensor ist auf der Platine untergebracht. Wie bei der Hauptplatine erfolgt die Spannungsversorgung über das externe Solarversorgungsmodul mit zwei 18650-Akkus. Zur sicheren Messung der Batterie- und Solarspannung werden wieder Impedanzwandler (OPVs) eingesetzt.

Zur vereinfachten Fehlersuche und Inbetriebnahme sind mehrere Testpunkte sowie Status-LEDs auf der Platine vorhanden.

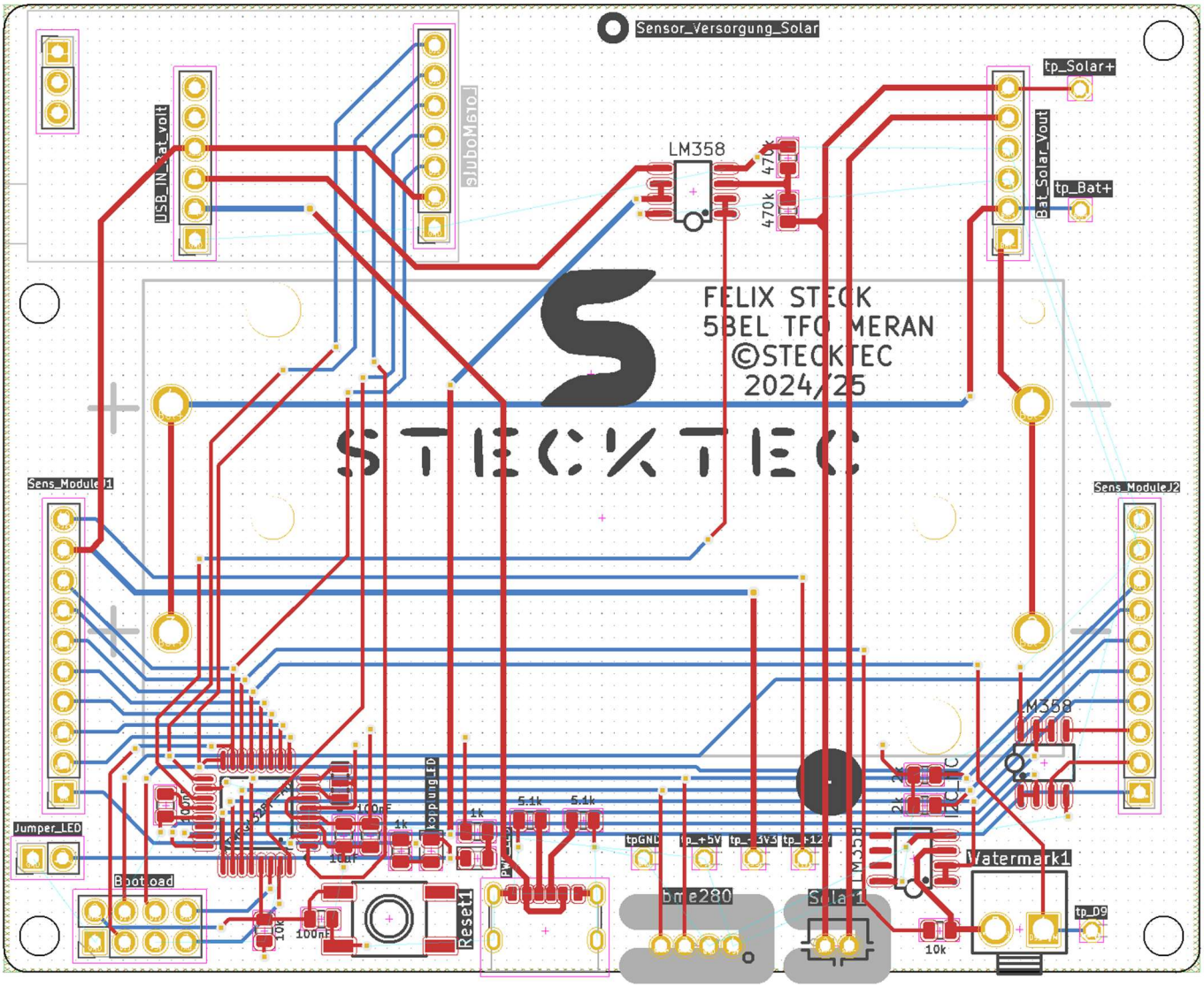
Für Erweiterungen verfügt die Sensorplatine über zwei Pinheader, auf die Zusatzmodule aufgesteckt werden können – zum Beispiel die Ventilsteuerungsplatine, mit der über eine H-Brücke (L298) zwei Magnetventile angesteuert werden können (Details siehe Abschnitt *Ventilsteuerung*). Die Belegung der Pinheader wurde bewusst so gestaltet, dass auch beliebige andere Erweiterungen realisierbar sind. Insgesamt stehen 20 Pins zur Verfügung, darunter verschiedene Versorgungsspannungen (+3.3 V, +5 V, +12 V, GND), Analogeingänge, Digitale Ein-/Ausgänge sowie I²C.

Darüber hinaus besitzt die Platine einen USB-Anschluss, der optional zur Aufladung der Akkus verwendet werden könnte – diese Funktion wurde jedoch auf der aktuellen Version des Solarversorgungsmoduls noch nicht implementiert. Da die Komplexität dieser Platine etwas geringer als die Hauptplatine ist, reichte ein 4-Layer-Stackup aus. Dabei wurden die beiden inneren Layer für GND und +5 V verwendet, um die äußeren Layer für Signalleitungen sowie die +3.3 V Versorgung freizuhalten.

## Komplette Schematic Sensorplatine



## Komplette Sensorplatine






## 5.3 Magnetventilsteuerung Platine

Die Magnetventilsteuerung-Erweiterungsplatine wurde entwickelt, um die Steuerung von zwei 12-Volt-Magnetventilen zu ermöglichen und ist eine Erweiterung für die Sensorplatine. Herzstück der Schaltung ist der Dual-H-Brücken-Treiber L298P, der die beiden Magnetventile unabhängig voneinander ansteuert. Die Steuerung erfolgt über vier digitale Pins (Vent1_D1, Vent1_D2, Vent2_D1, Vent2_D2) sowie ein zusätzliches Enable-Signal. Die Ansteuerlogik erlaubt das gezielte Öffnen und Schließen der Ventile durch gezieltes Schalten der H-Brücke.

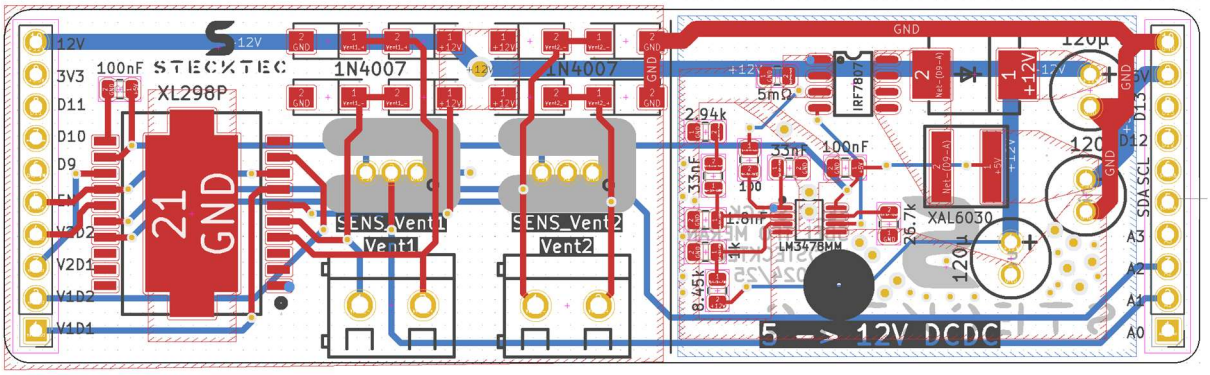
Um induktive Spannungsspitzen beim Abschalten der Magnetventile zu vermeiden, wurden zusätzlich zu den internen Schutzdioden des L298P externe Freilaufdioden (1N4007) verbaut.

Wie auch auf der Hauptplatine wurde der LM3478 als Step-Up-Wandler von 5 V auf 12 V verwendet, da für die Ansteuerung der Magnetventile eine Versorgungsspannung von 12 V erforderlich ist.

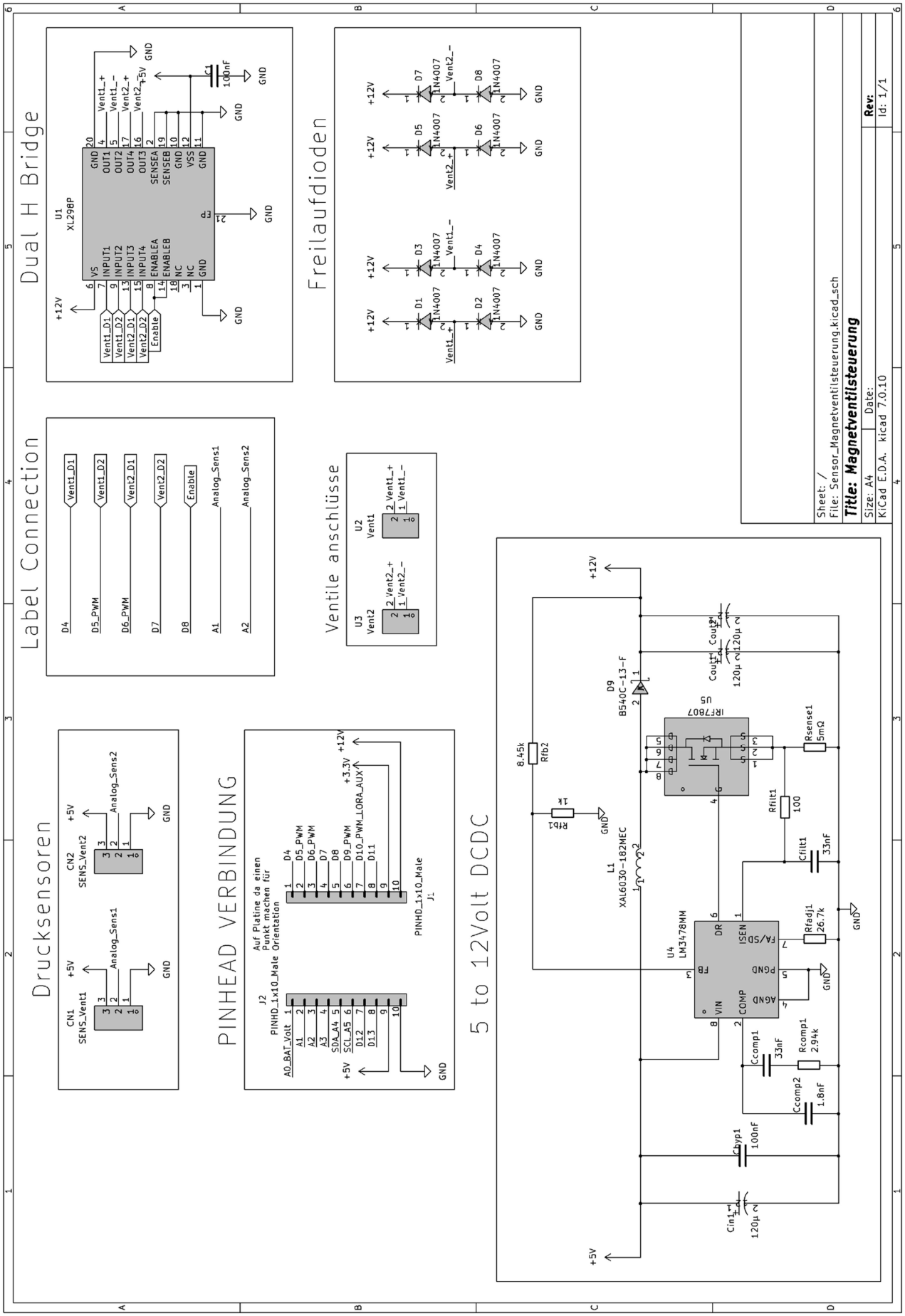
Die Verbindung zur Sensorplatine erfolgt über zwei 10-polige Pinheads, über welche neben den Steuersignalen auch alle nötigen Versorgungsspannungen (+3.3 V, +5 V, +12 V und GND), sowie analoge und digitale Signale übertragen werden.

Auch auf dieser Platine ist noch ein Durflusssensor für jedes Ventil verbaut, welcher misst, ob das Ventil auch wirklich geöffnet/geschlossen wurde.

In Stück für Stück zusammengeschlossenen Feldern kommt es oft vor, dass sich Verschiedene Magnetventile an verschiedenen Ecken der Felder befinden, wofür eine Sensorplatine mit diesem Modul in Kombination mit der guten Funkreichweite von Lora perfekt geeignet ist.




Auf der Platine kann man die klare Trennung zwischen dem 5V → 12V DCDC (reche Seite) und der eigentlichen Magnetventilsteuerung links erkennen. Diese Trennung ist wichtig da ein DCDC immer Elektromagnetische Störungen erzeugt, welche die andere Schaltung potenziell negativ beeinflussen könnte.


Dual H Bridge
U1
XL298P
+12V
+5V
GND
Vent1_D1
Vent1_D2
Vent2_D1
Vent2_D2
Enable
Vent1_+
Vent1_-
Vent2_+
Vent2_-
100nF
Freilaufdioden
D1
D2
D3
D4
D5
D6
D7
D8
1N4007
Label Connection
D4
D5_PWM
D6_PWM
D7
D8
A1
A2
Analog_Sens1
Analog_Sens2
Ventile anschlüsse
U2
Vent1
U3
Vent2
Drucksensoren
CN1
SENS_Vent1
CN2
SENS_Vent2
PINHEAD VERBINDUNG
J2
PINHD_1x10_Male
AO_BAT_Volt
SDA_A4
SCL_A5
D12
D13
+3.3V
D9_PWM
D10_PWM_LORA_AUX
D11
Auf Platine da einen Punkt machen für Orientation
5 to 12Volt DCDC
L1
XAL6030-182MEC
D9
B540C-13-F
U5
IRF7807
Rsense1
5mΩ
Rfilt1
100
Cfilt1
33nF
U4
LM3478MM
Rfadj1
26.7k
Rfb1
1k
Rfb2
8.45k
Ccomp1
33nF
Rcomp1
2.94k
Ccomp2
1.8nF
Cbyp1
100nF
Cout1
120µ
Cin1
120µ
Sheet: /
File: Sensor_Magnetventilsteuerung.kicad_sch
Title: Magnetventilsteuerung
Size: A4
Date:
Rev:
KiCad E.D.A. kicad 7.0.10
Id: 1/1

## 5.4 Solarversorgungsplatine (V1/V2)

### 5.4.1 Solarversorgungsplatine V1

Zu Beginn legte ich beim Entwurf der Solarversorgungsplatine besonderen Wert auf einen möglichst geringen Ruhestrom, um die Akkulaufzeit zu maximieren. Da es kein fertiges Modul gab, das all meinen Anforderungen entsprach, entschied ich mich, die Schaltung selbst zu entwickeln. Grundlage dafür war ein Schaltplan, den ich in einem YouTube-Video fand. Diesen passte ich entsprechend an und integrierte ihn direkt in das Layout der Aufsteckplatine. Zusätzlich wurde ein 3,3 V LDO vorgesehen, um das LoRa-Modul und den ursprünglich geplanten ESP32-S3 mit Strom zu versorgen.

Ein wesentlicher Vorteil der Umsetzung als eigenständiges Modul war – wie bereits bei den anderen Unterplatinen – die Möglichkeit, das Modul unabhängig testen zu können. Darüber hinaus konnte dasselbe Modul sowohl auf der Hauptplatine als auch auf der Sensorplatine verwendet werden, was Entwicklungsaufwand und Komplexität reduzierte.

Das Design wurde so ausgelegt, dass die Akkus entweder über das Solarpanel oder alternativ über ein externes 5 V-Netzteil geladen werden können. Dadurch ist eine flexible Stromversorgung gewährleistet – sowohl für den Betrieb im Feld als auch für Tests oder initiales Aufladen.

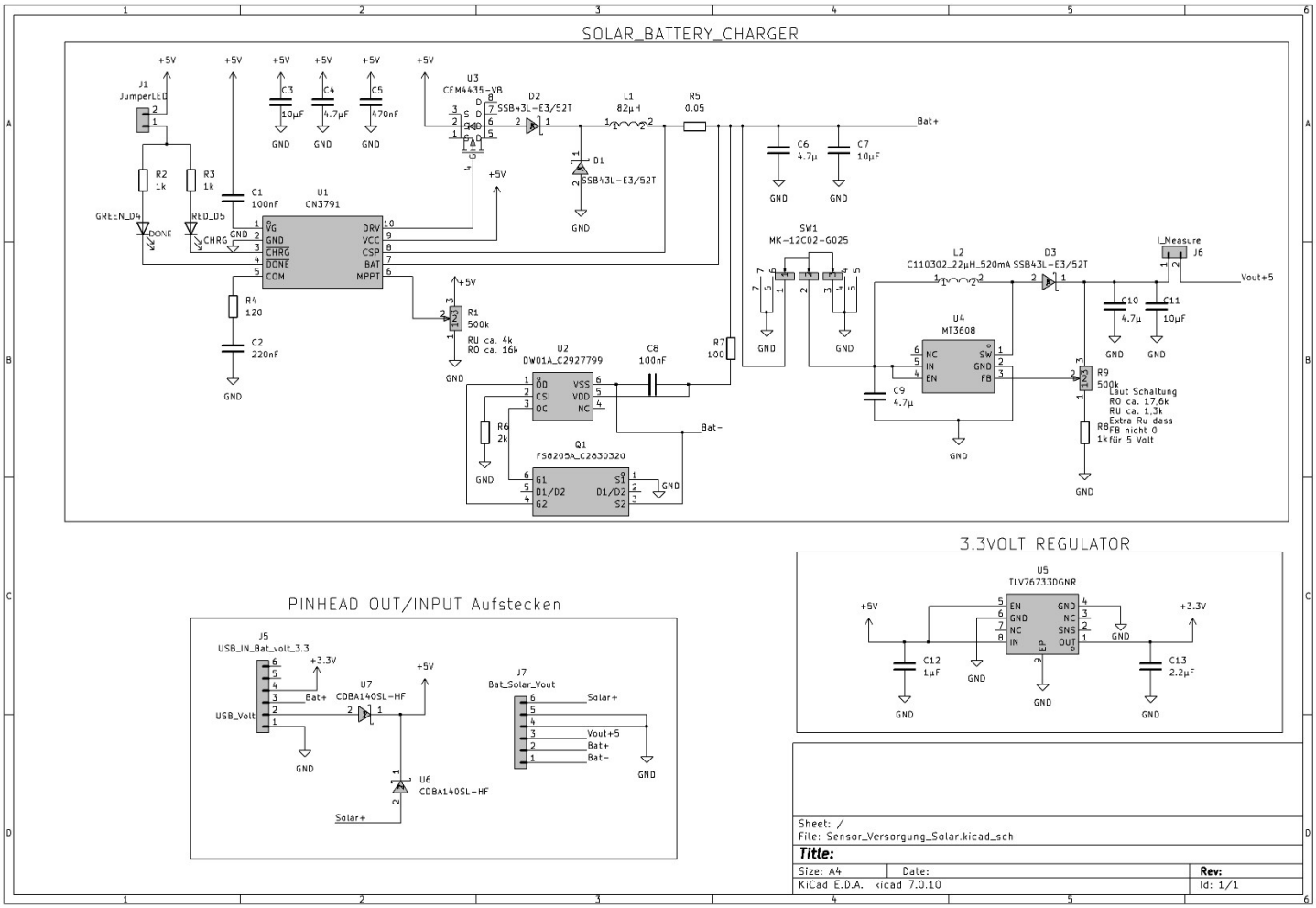
Auch bei dieser Platine wurde eine Markierung zur eindeutigen Orientierung beim Einstecken eingefügt. Für die 5 V-Versorgung wurde ein Step-Up-Wandler vom Typ MT3608 verwendet, bei dem über ein Potentiometer in der Feedback-Schleife die Ausgangsspannung präzise eingestellt werden kann.

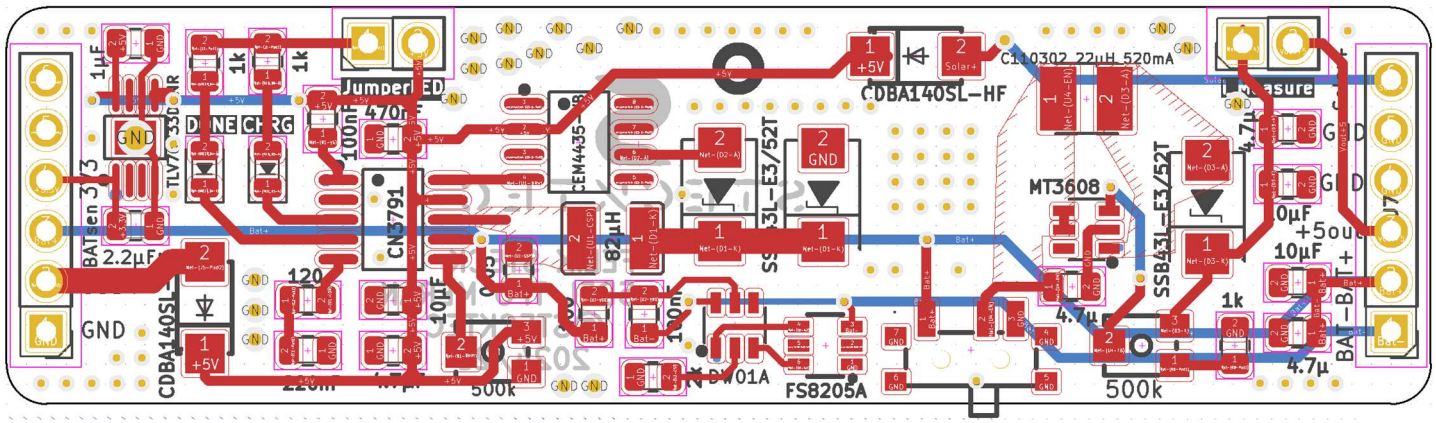
Um den Stromverbrauch im Idle-Zustand möglichst gering zu halten, wurden verschiedene Maßnahmen umgesetzt. Dazu zählen unter anderem die Verwendung möglichst großer Widerstandswerte bei Spannungsteilern und Vorwiderständen, um den Stromfluss zu minimieren. Außerdem wurden für die beiden Status-LEDs der Ladeelektronik Jumper vorgesehen, sodass diese im späteren Betrieb deaktiviert werden können, um weitere Einsparungen zu erzielen. Dieses Prinzip wurde auch bei den beiden Basisplatinen übernommen.

Zum Ein- und Ausschalten der Versorgung wurde ein kleiner SMD-Schiebeschalter integriert, über den der DCDC-Wandler vom Solarladeregler getrennt werden kann.

Nach dem Bestücken der Platine stellte sich jedoch heraus, dass der Laderegler (CN3791) nicht funktionierte – aller Wahrscheinlichkeit nach aufgrund von Layoutfehlern. Eine detaillierte Analyse zeigte zudem, dass auch der Schutz-IC DW01A nicht korrekt arbeitete. Aufgrund dieser Probleme entschied ich mich, eine neue, einfachere Schaltung für die zweite Version der Solarversorgungsplatine zu entwerfen.

## Schematic und PCB Solarversorgungsplatine V1



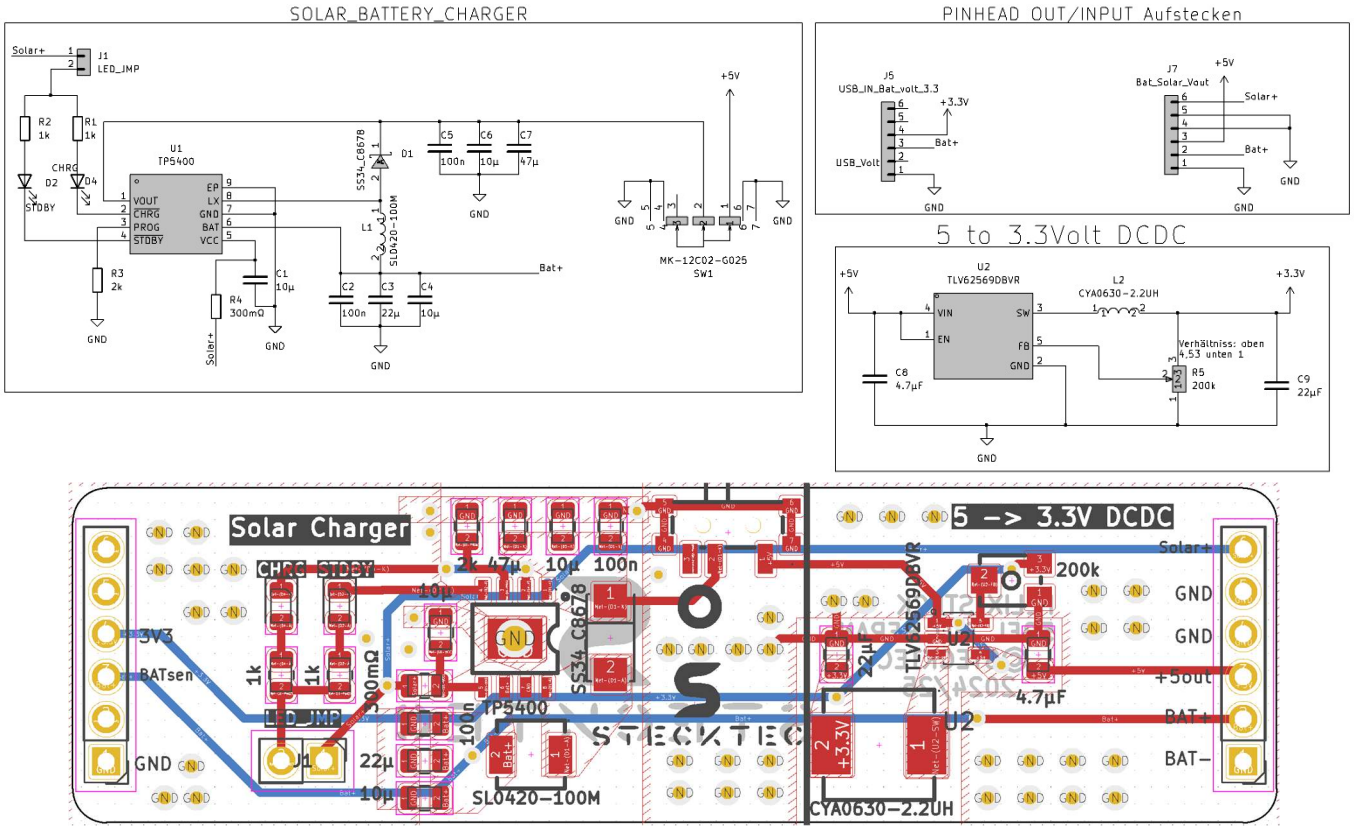
### 5.4.2 Solarversorgungsplatine V2

Aufgrund der zahlreichen Probleme mit der ersten Version war es bei der Entwicklung der V2-Platine oberste Priorität, möglichst schnell ein einfaches, aber zuverlässig funktionierendes System zu realisieren. Denn insbesondere durch die Schwierigkeiten mit dem SIM-Modul geriet ich zunehmend unter Zeitdruck.

Ich entschied mich daher für den TP5400, einen IC, den ich bereits in einem früheren Projekt erfolgreich eingesetzt hatte. Obwohl er nicht speziell für das Laden über Solar ausgelegt ist, bietet er den Vorteil, sowohl das Batterielade-Management als auch einen integrierten 5 V-Boost-Konverter in einem einzigen Baustein zu vereinen. Dadurch entfällt zusätzlicher Bauraum und die Schaltung bleibt kompakt.

Zusätzlich wurde ein separater 5 V-zu-3,3 V-Stepdown-DCDC-Wandler integriert, um gegenüber einem LDO eine höhere Energieeffizienz zu erreichen. Auch bei dieser Platine wurden Stromsparmaßnahmen berücksichtigt – obwohl der Ruhestrom des TP5400 standardmäßig etwas höher ist, weshalb ich auch zuerst versucht habe eine alternative Schaltung (V1) zu verwenden.

Ein entscheidender Aspekt bei der Verwendung dieses ICs ist das korrekte Platinen Layout. Aufgrund der hochfrequenten Komponenten des integrierten DCDC-Konverters ist es essenziell, kurze und möglichst gerade Leiterbahnen sowie großflächige Polygone zu verwenden. Das Datenblatt enthält einen empfohlenen Layoutvorschlag, den ich nahezu 1:1 übernommen habe. Dank der sorgfältigen Umsetzung dieser Vorgaben funktionierte die Platine direkt nach dem Bestücken fehlerfrei und lädt die Akkus zuverlässig.

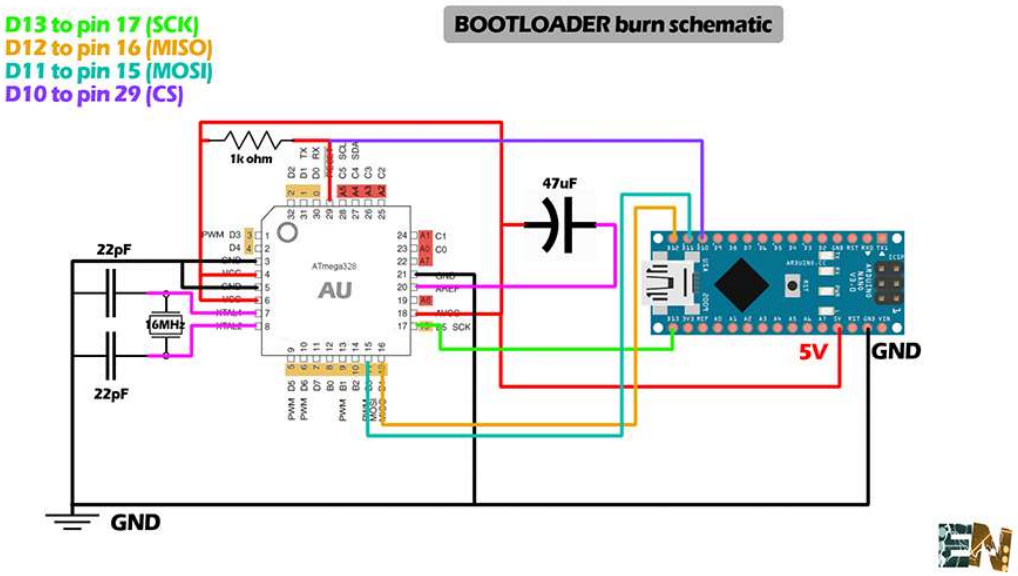


## 5.5 Programmiergerät

Um den Umgang mit dem ATmega Chip zu erleichtern, habe ich ein eigenes Programmiergerät entwickelt, das zwei zentrale Funktionen in einem kompakten Aufbau vereint: das Brennen des Bootloaders auf einen ATmega328P-Mikrocontroller und das anschließende Hochladen von Programmen. Durch dieses Gerät wird der Prozess zum Brennen des Bootloaders enorm vereinfacht und beschleunigt. Außerdem kann durch die Code Upload Funktion der platz und die kosten für die USB → TTL Converter Schaltung auf der zu programmierenden Platine eingespart werden, was sich vor allem bei einer Serienproduktion lohnen würde.

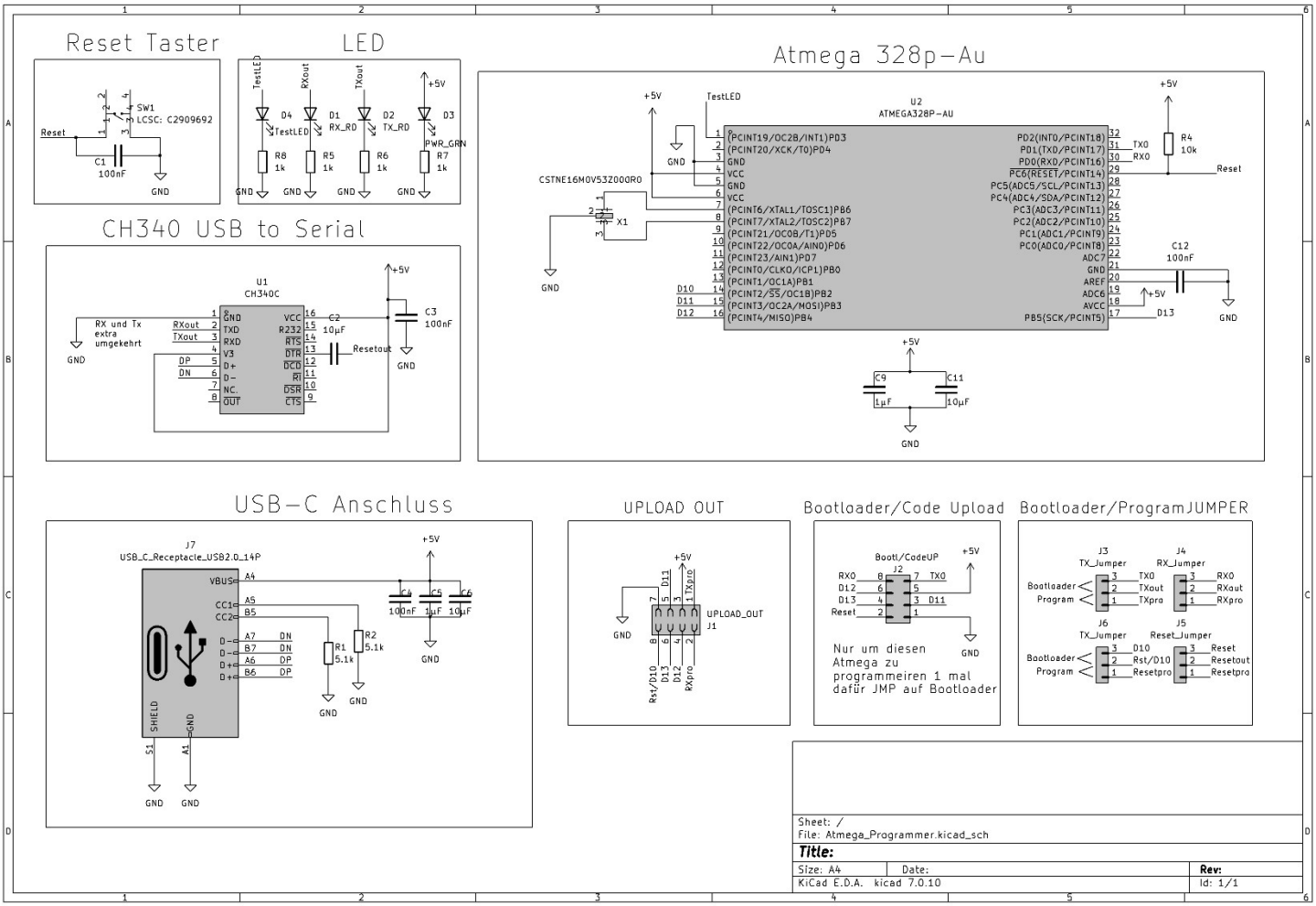
Das Programmiergerät wird über einen 8-poligen Pin-Header mit der zu programmierende Platine verbunden. Vier 3-polige Jumperleisten ermöglichen das Wechseln zwischen den beiden Betriebsmodi:

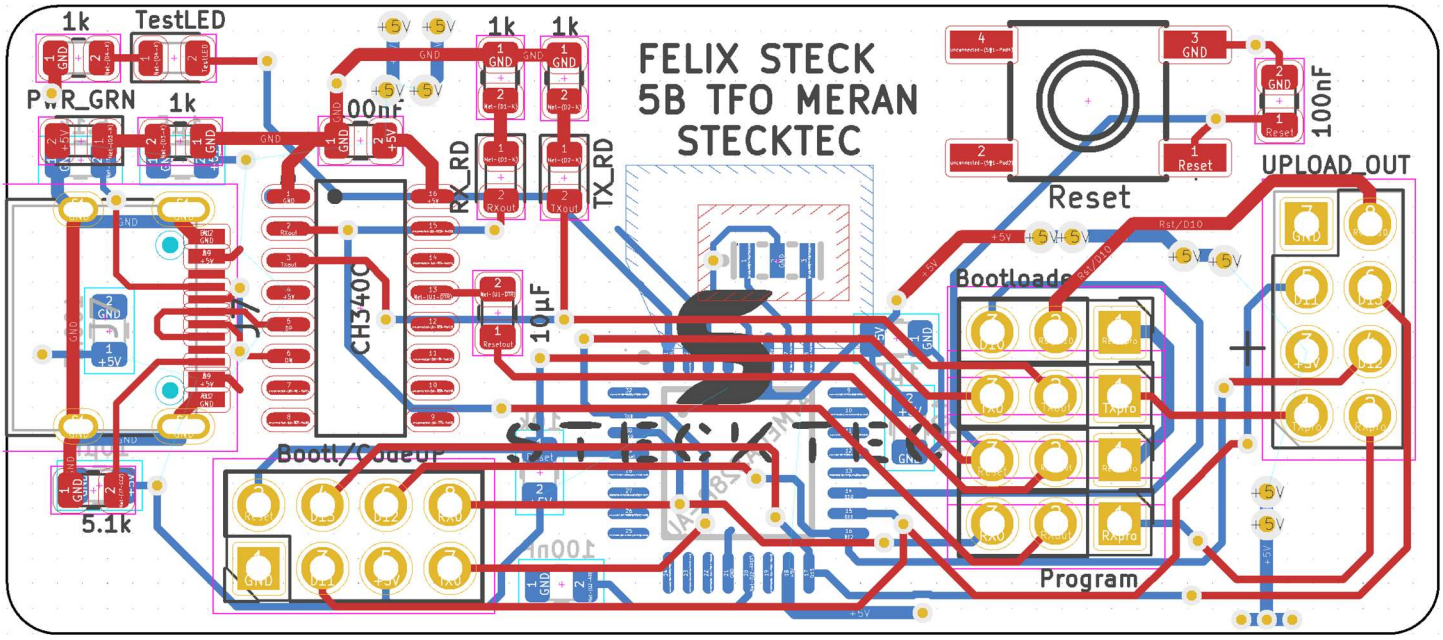
**Bootloader-Modus**: In dieser Einstellung fungiert das Gerät wie ein klassischer Arduino Nano. Es nutzt den integrierten Mikrocontroller, um den Bootloader über die SPI-Schnittstelle (MOSI, MISO, SCK, RESET) auf den Zielchip zu übertragen. Dafür müssten die Pins bei Verwendung eines noramlen Arduino Nanos folgendermaßen verbunden werden:




**Code-Upload-Modus**: Hier wird der interne Mikrocontroller des Programmiergeräts deaktiviert, und der USB-TTL-Konverter wird direkt mit dem Ziel Chip verbunden. So fungieren das Programmiergerät und der zu programmierende ATmega Chip auf der Platine wie ein kompletter Arduino Nano, welcher einfach über den USB-C Anschluss des Programmiergerätes an den PC angeschlossen werden, wo er auch als normaler Arduino Nano erkannt wird und problemlos programmiert, werden kann. Die Voraussetzung, dass das funktioniert, ist natürlich, dass der Bootloader bereits auf den Chip geflasht wurde.

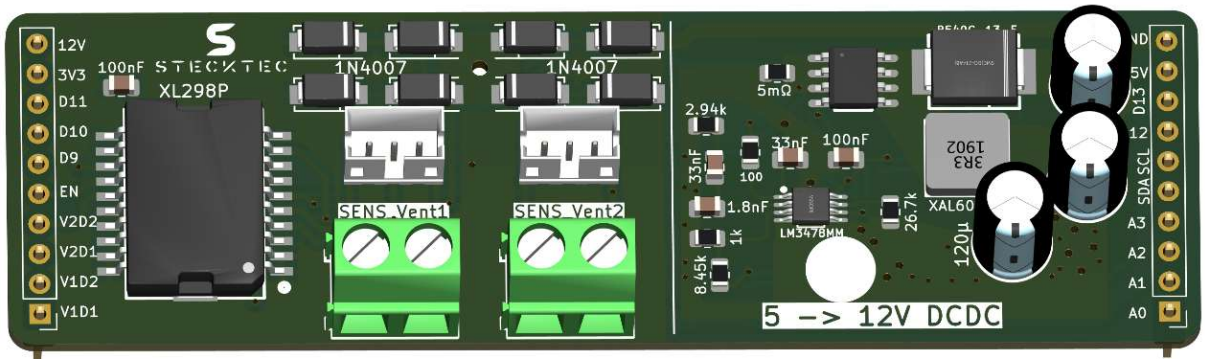
Die Platine des Programmiergeräts ist eine 2-Layer Platine, welche möglichst klein gehalten wurde. Um ein kleines handliches Programmiergerät zu realisieren. Außerdem verfügt sie über mehrere LEDs, um das Gerät einfach testen zu können und außerdem den Zustand des Programmierprozesses durch die RX und TX LEDs zu Visualisieren.



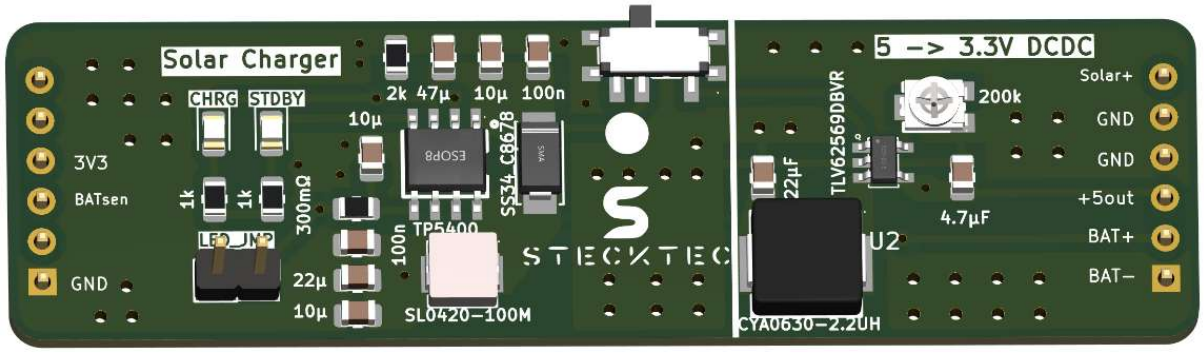



## 5.6 Platinen 3D-Modelle

### Magnetventilsteuereung Platine



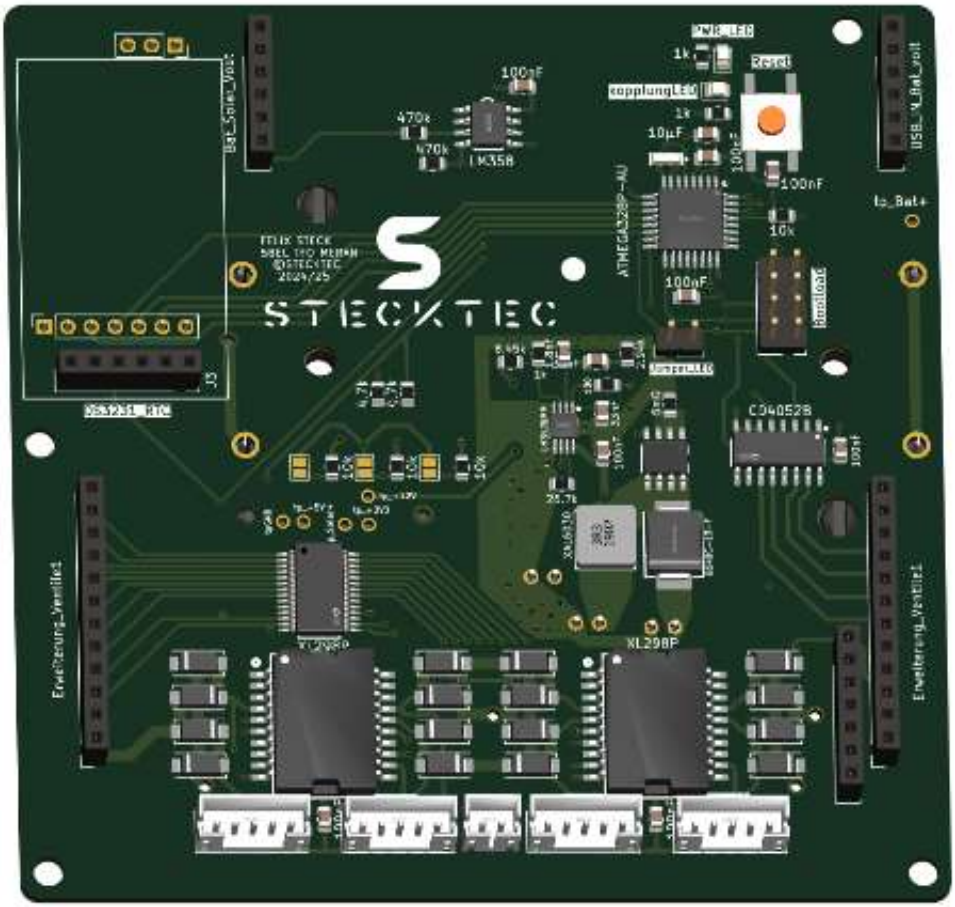

### Solarversorgung V2 Platine




### Programmiergerät Platine



**Hauptplatine V2**




# 6 Software

Bei der Software wurde darauf geachtet, den Code möglichst übersichtlich zu gestalten, indem er in verschiedene Funktionen aufgeteilt und ausgelagert wurde. Dadurch bleibt das Main File kompakt und gut lesbar.

In den folgenden Abschnitten beschreibe ich die komplexeren Teile des Codes. Auf die Verwendung der I²C-Geräte und das Auslesen der Analogsensoren gehe ich nicht näher ein, da sich diese Aufgaben mithilfe der jeweiligen Bibliotheken relativ einfach umsetzen ließen und dafür kein nennenswerter Code erforderlich war.

Für den BME280-Temperatur- und Luftfeuchtigkeitssensor wurde die **BME280_Light**-Library verwendet, die die Temperatur und Luftfeuchtigkeit direkt als Float-Werte zurückgibt.

Auch die Pins des Port-Expanders konnten mithilfe der **Adafruit_MCP23X17**-Library einfach angesteuert werden.

## 6.1 Ventilansteuerung

Die Magnetventile werden über H-Brücken angesteuert. Da ich relativ simple H-Brücken-ICs verwende, muss besonders darauf geachtet werden, dass nicht beide Steuerpins gleichzeitig auf High oder Low gesetzt werden, da dies zu einem Kurzschluss führen kann. Um das zu vermeiden, wird in der Funktion zuerst der Ausgang (Enable Pin) deaktiviert, bevor die Steuerpins umgeschaltet werden.

Da es sich um bistabile Ventile handelt, benötigen sie lediglich einen kurzen Spannungsimpuls, um ihren Zustand zu ändern. Danach halten sie ihre Position ohne weitere Energiezufuhr.

Um das Main File übersichtlich zu halten, habe ich eine eigene Funktion zur Ansteuerung der Magnetventile geschrieben, die das Umschalten im Main Code wesentlich einfacher und übersichtlicher macht. Der folgende Codeabschnitt zeigt diese Funktion. D1_pin und D2_pin sind die beiden Steuerleitungen der H-Brücke. Diese werden mit dem gewünschte Ventilzustand werden an die Funktion übergeben, die sich dann selbstständig um das Umschalten kümmert.

C++ Ventilansteuerung

```cpp
void updateVent(uint8_t D1_pin, uint8_t D2_pin, bool state)
{
    if (state == true)  // Ventil öffnen
    {
        // Sicherstellen, dass der Enable-Pin vor dem Schalten LOW ist
        digitalWrite(ENABLE_PIN, LOW);
        digitalWrite(D2_pin, LOW); // Beide Steuerpins zunächst auf LOW setzen
        digitalWrite(D1_pin, LOW);
        delay(10);  // Kurze Pause dass Mosfets komplett umschalten können
        digitalWrite(D2_pin, HIGH);
        digitalWrite(ENABLE_PIN, HIGH);
        delay(10);  // Spule kurz aktiv lassen

        digitalWrite(ENABLE_PIN, LOW); // Ausgang wieder deraktivieren → Energie sparen
    }
    else if (state == false)  // Ventil schließen
    {
        digitalWrite(ENABLE_PIN, LOW);
        digitalWrite(D1_pin, LOW);
        digitalWrite(D2_pin, LOW);
        delay(10);
        digitalWrite(D1_pin, HIGH);
        digitalWrite(ENABLE_PIN, HIGH);
        delay(10);
        digitalWrite(ENABLE_PIN, LOW);
    }
}
```

## 6.2 Sim-Kommunikation

Die Internetkommunikation des EIC-Bewässerungssystems erfolgt über das SIM7000G-Modul, welches über die UART-Schnittstelle an den Arduino auf der Hauptplatine angebunden ist. Als Bibliothek kommt TinyGsmClient zum Einsatz, die speziell für GSM- und LTE-Module auf Embedded-Systemen wie dem Arduino entwickelt wurde. Diese ermöglicht die Nutzung von Mobilfunkverbindungen und stellt Funktionen zur Verfügung, um über Standardprotokolle wie HTTP mit Servern zu kommunizieren.

Die Kommunikation dient insbesondere der bidirektionalen Übertragung von Daten mit der Firebase-Datenbank, wobei sowohl Sensorwerte in der Datenbank aktualisiert werden können, aber auch der Zustand der Ventile ausgelesen werden kann. Die Kommunikation erfolgt dabei über einen zwischengeschalteten Proxy, welches unter einer festen IP-Adresse erreichbar ist.

Ich habe die beiden Funktionalitäten (senden / empfangen) in 2 Funktionen aufgeteilt:

- **putValue(path, value)**: sendet einen HTTP-PUT-Request, um einen bestimmten Datenpfad auf dem Server mit einem neuen Wert zu aktualisieren.
- **getValue(path)**: führt einen HTTP-GET-Request aus, um den aktuellen Wert eines Datenpfads vom Server zu lesen.

Dabei wird die TinyGsmClient-Verbindung jeweils kurzzeitig aufgebaut, genutzt und anschließend wieder getrennt. Diese Maßnahme spart natürlich viel Strom, was im Anbetracht des Batteriebetriebs natürlich wichtig ist.

### 6.2.1 Aufbau einer HTTP-Request

Das verwendete Protokoll ist HTTP/1.1 (Hypertext Transfer Protocol), ein Protokoll, das auf dem TCP/IP-Stack aufsetzt. Bei einem HTTP-Request wird typischerweise eine Zeile mit der Methode (z. B. GET, PUT, POST) und dem Pfad gesendet, gefolgt von Header-Informationen (z. B. Host, Content-Type, Content-Length) und – bei Methoden wie PUT – einem Nachrichtentext (Payload).

Erzeugung der PUT Request:

C++ PUT-Erzeugen

```cpp
client.print("PUT /proxy?path=");
client.print(path);
client.println(" HTTP/1.1");
client.println("Host: 13.50.73.117");
client.println("Content-Type: text/plain");
client.print("Content-Length: ");
client.println(value.length());
client.println("Connection: close");
client.println();
client.println(value);
```

Beispiel einer über die Software erzeugten PUT-Request:

C++ PUT-Request

```
PUT /proxy?path=/sensor/feuchtigkeit HTTP/1.1
Host: 13.50.73.117
Content-Type: text/plain
Content-Length: 4
Connection: close

45.6  //Payload
```

Nach dem Absenden der Request wartet die Software auf eine Antwort des Servers. Um Zeitüberschreitungen zu vermeiden, ist ein Timeout-Mechanismus eingebaut.

C++ Timeout -Mechanismus

```cpp
unsigned long timeout = millis();
while (client.connected() && !client.available())
{
    if (millis() - timeout > 10000)
    {
        // PUT Response timeout
        client.stop();
        return;
    }
}
```

### 6.2.2 Entwicklung und Herausforderungen

Ursprünglich war geplant, die Kommunikation ohne Library direkt über AT-Kommandos zu implementieren. Aufgrund zahlreicher Probleme – wie unzuverlässige Antworten, fehleranfällige Befehlsabfolgen und Verbindungsabbrüche – wurde nach mehreren Versuchen die TinyGsmClient-Bibliothek eingesetzt, die sich als deutlich robuster und zuverlässiger erwies.

Die Entwicklung dieses Teils war eine der komplexesten Phasen im Projekt und erstreckte sich über mehrere Monate, was auch den Zeitplan etwas durcheinanderbrachte. Erst nach umfassender Recherche und Debugging konnte eine stabile Anbindung an die Firebase-Datenbank realisiert werden. Der funktionierende Internetzugang war ein zentraler Meilenstein für den Fortschritt des Projekts.

## 6.3 Lora Kommunikation

Die Kommunikation zwischen der Hauptplatine und der Sensorplatine erfolgt über das LoRa E32-Modul und ist bidirektional durch die sogenannte Master-Slave-Kommunikation. Das bedeutet, die Hauptplatine sendet ein Datenpaket, auf das das Sensor-Modul mit einer Antwort reagiert. Sollte die Hauptplatine innerhalb von 5 Sekunden keine Antwort erhalten, wird das Datenpaket automatisch erneut gesendet. Dieser Wiederholungsmechanismus sorgt dafür, dass die Kommunikation stabil bleibt.

Die Sensorplatine sendet ein Datenpaket nur dann, wenn sie zuvor ein Datenpaket von der Hauptplatine empfangen hat. Dieses Kommunikationssystem hilft, den Stromverbrauch der Sensorplatine erheblich zu minimieren, da sie keine Daten überträgt, solange sie keine empfängt.

Die Hauptplatine sendet alle 5 Sekunden ein JSON-formatiertes Paket, das u. a. die Ventilzustände ("V1SState" und "V2SState") enthält. Die Sensorplatine verarbeitet diese Informationen, steuert dementsprechend ihre Ventile an und antwortet mit einem eigenen JSON-Paket. Dieses enthält dann die Sensordaten Temperatur ("tmp"), Luftfeuchtigkeit ("hum"), Drucksensorstatus ("ps1", "ps2") und den aktuellen Watermark-Bodenfeuchtigkeits-Wert ("wtm").

Der folgende Codeteil zeigt die zentrale LoRa-Kommunikationsfunktion sendWithResponse() auf der Hauptplatine:

C++ Lora Funktion Hauptplatine

```cpp
void sendWithResponse(const String &message)
{
    // Senden der Nachricht über das LoRa-Modul
    ResponseStatus sendStatus = e32ttl100.sendMessage(message);
    Serial.print("Sent: ");
    Serial.println(message);

    unsigned long responseStart = millis();
    bool responseReceived = false;

    // Warten auf eine Antwort für maximal 4 Sekunden
    while (millis() - responseStart < 4000)
    {
        if (e32ttl100.available())  // Wenn eine Antwort verfügbar ist
        {
            rc = e32ttl100.receiveMessage();   // Nachricht empfangen

            // Wenn der Empfang erfolgreich war (Statuscode 1 = OK)
            if (rc.status.code == 1)
            {
                Serial.print("Received response: ");
                Serial.println(rc.data);  // Antwortinhalt ausgeben
                responseReceived = true;
                break;  // Schleife verlassen, da Antwort empfangen wurde
            }
        }
    }

    if (!responseReceived) // Wenn keine Antwort empfangen wurde
    {
        Serial.println("No response received");
    }
}
```

## 6.4 Smartphone App

Zur einfachen Bedienung des EIC-Systems wurde eine eigene Smartphone App mit React Native entwickelt. Sie ermöglicht die Verwaltung der Wiesen, die Steuerung der Ventile sowie die Anzeige von Sensordaten. Die App ist benutzerfreundlich gestaltet und in verschiedene Tabs unterteilt, die jeweils eine spezifische Funktion übernehmen.

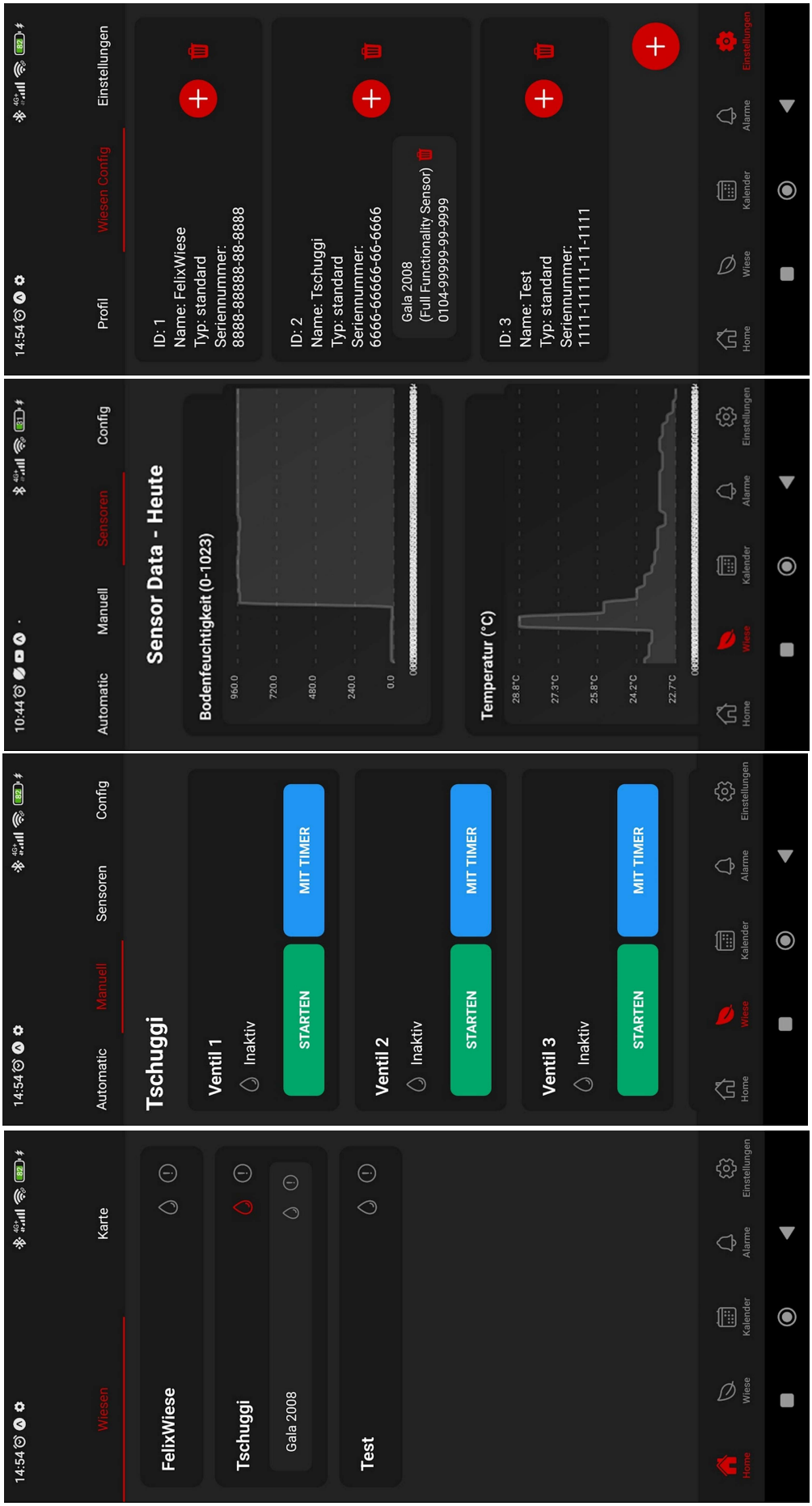
Im Tab „Wiesen Config“ können neue Wiesen angelegt und bestehende verwaltet werden. Jeder Eintrag enthält eine eindeutige ID, einen frei wählbaren Namen und die Seriennummer der dazugehörigen Hauptplatine.
Zu jeder Wiese (Hauptmodul) können bis zu fünf Sensormodule hinzugefügt werden, welche unterschiedliche Funktionalitäten besitzen – z. B. nur Bodenfeuchtigkeit, nur Ventilsteuerung oder auch alle bisher realisierten Funktionen (Bodenfeuchte, Temperatur, Luftfeuchtigkeit und zwei Magnetventile).

Sobald eine neue Wiese im Konfigurations-Screen angelegt wurde, erscheint sie auch im „Wiesen“-Tab, wo eine Gesamtübersicht über alle Wiesen und deren Sensormodule angezeigt wird. Durch ein Rufzeichen-Symbol ist sofort erkennbar, bei welcher Wiese ein Problem vorliegt – es leuchtet entsprechend auf. Durch Antippen einer beliebigen Wiese gelangt man in den Detailbereich dieser Wiese. Dort gibt es mehrere Unter-Screens:

- **Manuell**: Hier ist es möglich, die Ventile manuell ein- und auszuschalten oder sie mittels eines Timers für eine bestimmte Zeit zu aktivieren. Dies ist besonders nützlich für erste Tests oder um beispielsweise ein Leck im Tropfschlauch zu lokalisieren.
- **Sensoren**: Dieser Bereich zeigt – sofern vorhanden – die Sensorwerte als Diagramme an.
- **Automatik** (noch in Entwicklung): Dieser Bereich soll zukünftig ermöglichen, die Bewässerung anhand eines Zeitplans zu steuern oder automatisch aufgrund der Bodenfeuchtigkeit den optimalen Bewässerungszeitpunkt zu bestimmen.

Für die Zukunft sind weitere Funktionen wie Alarme oder ein Kalender vorgesehen. Diese waren jedoch nicht Teil der ursprünglichen Projektziele.

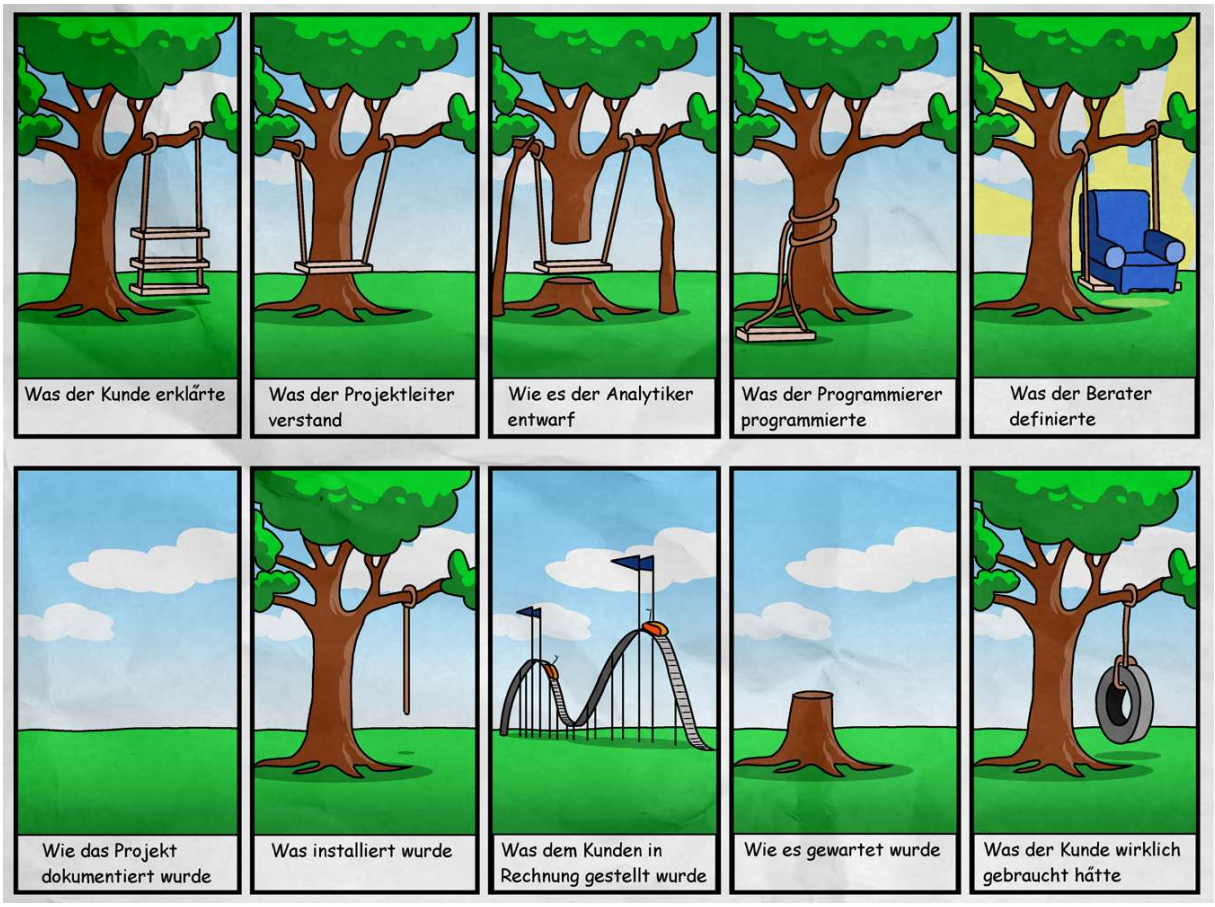
Die App kommuniziert mit der Firebase-Datenbank, wobei alle Zustände, Konfigurationen und Sensordaten laufend synchronisiert werden. Änderungen über die App werden über das SIM7000-Modul umgehend an die Hardware weitergegeben.


14:54
Wiesen
Karte
FelixWiese
Tschuggi
Gala 2008
Test
Home
Wiese
Kalender
Alarme
Einstellungen
Automatic
Manuell
Sensoren
Config
Tschuggi
Ventil 1
Inaktiv
STARTEN
MIT TIMER
Ventil 2
Inaktiv
STARTEN
MIT TIMER
Ventil 3
Inaktiv
STARTEN
MIT TIMER
10:44
Automatic
Manuell
Sensoren
Config
Sensor Data - Heute
Bodenfeuchtigkeit (0-1023)
960.0
720.0
480.0
240.0
0.0
Temperatur (°C)
28.8°C
27.3°C
25.8°C
24.2°C
22.7°C
14:54
Profil
Wiesen Config
Einstellungen
ID: 1
Name: FelixWiese
Typ: standard
Seriennummer:
8888-88888-88-8888
ID: 2
Name: Tschuggi
Typ: standard
Seriennummer:
6666-66666-66-6666
Gala 2008
(Full Functionality Sensor)
0104-99999-99-9999
ID: 3
Name: Test
Typ: standard
Seriennummer:
1111-11111-11-1111

# 7 Projektmanagement

Projektmanagement ist ein entscheidender Erfolgsfaktor in jedem größeren Vorhaben – sei es in der Wirtschaft, im technischen Bereich oder in der Schule. Es sorgt dafür, dass Ziele klar definiert, Aufgaben sinnvoll verteilt, Ressourcen effizient genutzt und zeitliche Abläufe realistisch geplant werden. Ohne strukturiertes Projektmanagement entstehen leicht Missverständnisse, Doppelarbeit, Zeitverluste und am Ende möglicherweise ein Ergebnis, das nicht den ursprünglichen Erwartungen entspricht.

Das folgende Bild bringt diese Problematik auf humorvolle Weise auf den Punkt:




Es zeigt, wie unterschiedlich ein und dieselbe Idee von verschiedenen Projektbeteiligten interpretiert, umgesetzt und dokumentiert wird – und wie groß die Abweichung zwischen dem ursprünglichen Wunsch des Kunden und dem finalen Ergebnis sein kann. Genau hier setzt gutes Projektmanagement an: Es stellt sicher, dass alle Beteiligten dasselbe Ziel verfolgen, der Fortschritt regelmäßig überprüft wird und das Endprodukt dem tatsächlichen Bedarf entspricht.

Kurz gesagt: Projektmanagement schafft Struktur, Klarheit und Transparenz – und verhindert, dass Projekte „aus dem Ruder laufen“.

## 7.1 Projektantrag

Damit ein Projekt genehmigt werden kann, muss jeder Schüler einen Projektantrag verfassen. In diesem Antrag werden die angestrebten Ziele des Projekts festgehalten. Am Ende wird gemeinsam mit den betreuenden Lehrpersonen eine maximal erreichbare Note für das Projekt festgelegt. Diese Note kann – abhängig vom tatsächlichen Umsetzungsgrad – erreicht werden. Dieses Vorgehen soll verhindern, dass es am Ende des Schuljahres zu Unstimmigkeiten kommt, und gewährleistet, dass das Projekt im Verlauf des Schuljahres nicht wesentlich verändert wird.

# Projektantrag

Seite 1/2

Ansuchen für ein Maturaprojekt, Fassung September 2024

| | | |
|---|---|---|
| Projekttitel: | Stecktec EIC | |
| Projektleiter: | Felix Steck | |
| Klasse und Schuljahr: | 5B EL | Schuljahr 2024 / 2025 |

Logo des Projektes:

STECKTEC

Kurze Beschreibung des Projektes: Das Ziel ist eine vielseitig einsetzbare modulare Bewässerungsanlage zu bauen, welche es erlaubt, basierend auf verschiedene Sensorwerten und eingestellten Daten in der Handyapp, die Bewässerung von verschiedenen Kulturen in der Landwirtschaft zu Steuern.

Hauptstation: Sim Modul zur Verbindung mit Server für Steuerung mit Handyapp,
Lora Modul zu senden/empfangen von Daten der Sensoren ,
6 Relaismodule zur Ansteuerung von Magnetventilen, mit 6 sensoren die messen
ob die ventile wirkich offen/zu sind
mit ESP32-S3 Wroom Modul
mit Solar und 12V Batterie

Sensoren: Platine mit Arduino Nano Chip,
Lora Sender,
Auf Low Power achten,
Stromversorgung 18650 Akku mit Solarzelle/ Laderegler + 5V DCDC
Entweder Module für verschiedene Sensoren zum aufstecken oder alles auf der Platine,
Verschiedene Sensorarten: Bodenfeuchtesensor, Temp/Luftfeuchte Sensor, doppeltes Relaismodul
Zur ansteuerung von externen Ventilen

Benotungsrelevante Meilensteine des Projektes:

- Meilenstein 1: Platine Sensoren in PW9 (1. Semester)
- Meilenstein 2: Hauptplatinenabgabe in PW9 (1. Semester)
- Meilenstein 3: Platinen fertig gelötet in PW16 (1. Semester)
- Meilenstein 4: Kommunikation Hauptplatine ←→ Sensoren funktioniert PW22(2. Semester)
- Meilenstein 5: Hauptplatine←→Handyapp daten senden funktioniert >PW24(2. Semester)
- Meilenstein 6: Abgabe des Projektes in PW30 (2. Semester)
- Meilenstein 7: Abgabe der Projektdokumentation in PW34 (2. Semester)

# Projektantrag

Seite 2/2

| | | | |
|---|---|---|---|
| Anhänge: | ☒ JA | ( 2 ) Seiten | ☐ NEIN |
| Projekt genehmigt: | ☒ JA | | ☐ NEIN |

Maximal erreichbare Note bei 100% Funktionalität:
(unter Berücksichtigung der unten genannten Präzisierungen) 10 (zehn)

Präzisierungen des PAG:

- Note je nach Funktionalität

PAG 1: Martin De Tomaso

PAG 2: Ivan Huber

Technologie & Projektmanagement Steckbrief Systeme
OK

Der Projektleiter

Felix Steck

## 7.2 Pflichtenheft

Im Pflichtenheft sollen die Schüler ihr geplantes Projekt so konkret wie möglich im Voraus beschreiben, eventuelle Ziele und Nicht-Ziele werden erwähnt. Es werden alle Materialien und Bauteile, die voraussichtlich verwendet werden, aufgelistet. Dadurch können die Lehrpersonen den Umfang, die Komplexität und dadurch auch die voraussichtliche Note für das Projekt bestimmen und es besser verstehen.

# Pflichtenheft STECKTEC EIC

Das Ziel ist eine vielseitig einsetzbare modulare Bewässerungsanlage zu bauen, welche es erlaubt, basierend auf verschiedene Sensorwerte und eingestellten Daten in der Handyapp, die Bewässerung von verschiedenen Kulturen in der Landwirtschaft zu Steuern.

Diese besteht aus einer Hauptstation und mehreren Ausenstationen (Sensoren/Aktoren)

Ich baue eine Temperatur/Lufteuchtigkeitssensor, einen Bodenfeuchte Sensor und einen Aktor mit 2 Relais welcher 2 Magnetventile ansteuern kann.

**Hauptstation:** Sim Modul zur Verbindung mit Server für Steuerung mit Handyapp,
Lora Modul zu senden/empfangen von Daten der Sensoren (voraussichtlich Ebyte E32 Modul),
Ein RTC Modul zur Bestimmung der aktuellen Zeit,
6 Relaismodule zur Ansteuerung von Magnetventilen,
vielleicht auch 6 Druck/Durchflusssensoren zum Messen ob Ventile wirklich offen/zu sind,
mit ESP32-S3 Wroom Modul
Soll über Solar und eine 12Volt Batterie betrieben werden Laderegler wahrscheinlich fertige Platine
Die Haputstation soll ein 3D gedrucktes Gehäuse haben oder alternativ in eine normale Verteilerdose eingebaut werden.

**Handyapp:** Die Daten sollen von der Hauptstation zu einer Firebase Datenbank gesendet werden,
diese wir dann wiederum über eine mit ReactNative Programmierte Handyapp ausgelesen / geändert
Die App soll alle Sensorwerte anzeigen können, Die Relais manuell schalten, man kann abhängig vom Datum einen automatischen Bewässerungszyklus einrichten, Man kann die Bewässerung anhand der Sensorwerte anpassen (länger oder weniger lange bewässern).

**Sensoren allgemein:**
Platine mit Arduino Nano Chip,
Lora Sender (voraussichtlich Ebyte E32 Modul),
Die Sensor Platinen müssen natürlich relativ stromsparend sein da sie Batteriebetrieben sind,
Stromversorgung 18650 Akku mit Solarzelle/ Laderegler + 5V DCDC
Entweder Module für verschiedene Sensoren zum Aufstecken oder alles auf der Platine,
Die Sensoren sollen ein 3D gedrucktes Gehäuse haben oder alternativ in eine normale Verteilerdose eingebaut werden.

**Temperatur/ Luftfeuchtigkeitssensor:**

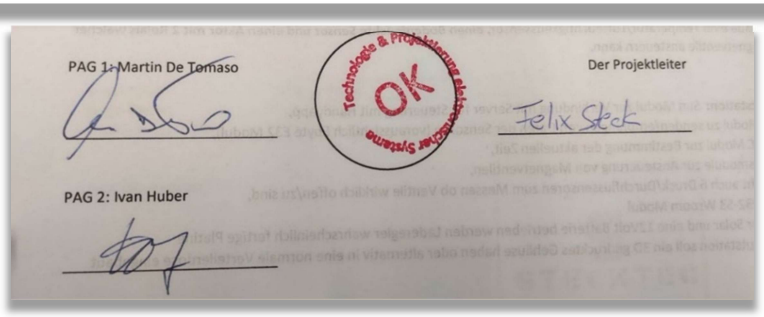
Für den Temp/Luftfeuchtigkeitssensor verwende ich einen BME280 Sensor welcher die Daten misst.

**Bodenfeuchtesensor:**

Als Bodenfeuchtesensor verwende ich den 200SS WATERMARK Sensor, dieser ist im vergleich zu den herkömmlichen Kapazitiven Bodenfeuchtesensoren genauer und viel wichtiger, er misst die sogenannte Saugspannung mit der die Pflanzen Feuchtigkeit anziehen und nicht den generellen Feuchtigkeitsgehalt der Erde

**Ventilsteuerung:**

Die Ventilsteuerung hat einfach wie auch die Hauptstation 2 Relais und vielleicht 2 Druck/Durchflusssensoren. zum Messen ob Ventile wirklich offen/zu sind



## 7.3 Beschreibung der Working Packages

Ein Working Package ist eine klar abgegrenzte, überschaubare Einheit innerhalb eines Projekts. Es beschreibt eine konkrete Aufgabe oder einen Aufgabenbereich, der innerhalb eines bestimmten Zeitrahmens abgeschlossen werden soll. Working Packages helfen dabei, ein komplexes Projekt in kleinere, besser plan- und kontrollierbare Schritte zu unterteilen.

Jedes Arbeitspaket hat ein konkretes Ziel, oft mit einem bestimmten Ergebnis (z. B. ein Schaltplan, ein Testprogramm oder eine fertige Platine). Es ist die Basis für die Projektplanung, Zeitplanung und auch für die Dokumentation.

**Meine Working Packages:**

WP1: Projektdefinition → Ausdenken des Projekts

WP2: Erstellen Pflichtenheft

WP3: Recherche/Testbauteile → Test des LoRa-Moduls, Temperatursensors und SIM-Moduls

WP4: Planung Schaltplan Sensor → Planung des Schaltplans, Überlegung der Bauteile, Skizze…

WP5: Zeichnen Schaltpläne Sensor → Aus fertigem Konzept einen fertigen Schaltplan zeichnen

WP6: Konzipierung Hauptplatine → Was braucht es auf der Hauptplatine, Skizze

WP7: Schaltplan Hauptplatine → Aus fertigem Konzept einen fertigen Schaltplan zeichnen

WP8: Zeichnen der Platinen → Mit dem fertigen Schaltplan Routen beginnen, am Ende die fertigen Gerber-Files nach JLC senden

WP9: Bestellung Bauteile → Bestellung aller benötigten Bauteile für die Platinen bei LCSC

WP10: Start Software Sensoren → Software für die Sensorplatine, damit ich, sobald die Platine ankommt, gleich ein Testprogramm hochladen kann und die Sensoren ausgelesen werden können

WP11: Löten der Platinen → Platinen komplett, alle Bauteile löten

WP12: Test der Sensorplatine → Testprogramm aus WP10 hochladen, es soll funktionieren

WP13: Programmierung Sensorplatine → Weiterentwicklung Programm, bis LoRa funktioniert

WP14: Überarbeitung Sensorplatine → Mögliche Fehler ausbessern, Bestellung und Bestücken

WP15: Test Hauptplatine → Upload von Testcode auf Hauptplatine, damit das Relais-Schalten funktioniert

WP16: Programmierung Hauptplatine → Bis LoRa-Kommunikation funktioniert

WP17: Überarbeitung Hauptplatine → Wenn nötig, Ausbesserung von Fehlern, Bestellung und Bestücken

WP18: SIM-Programmierung → Programmierung, bis SIM-Modul Daten an Datenbank sendet

WP19: Programmierung Handy-App → Handy-App programmieren, die mit Hauptplatine kommuniziert

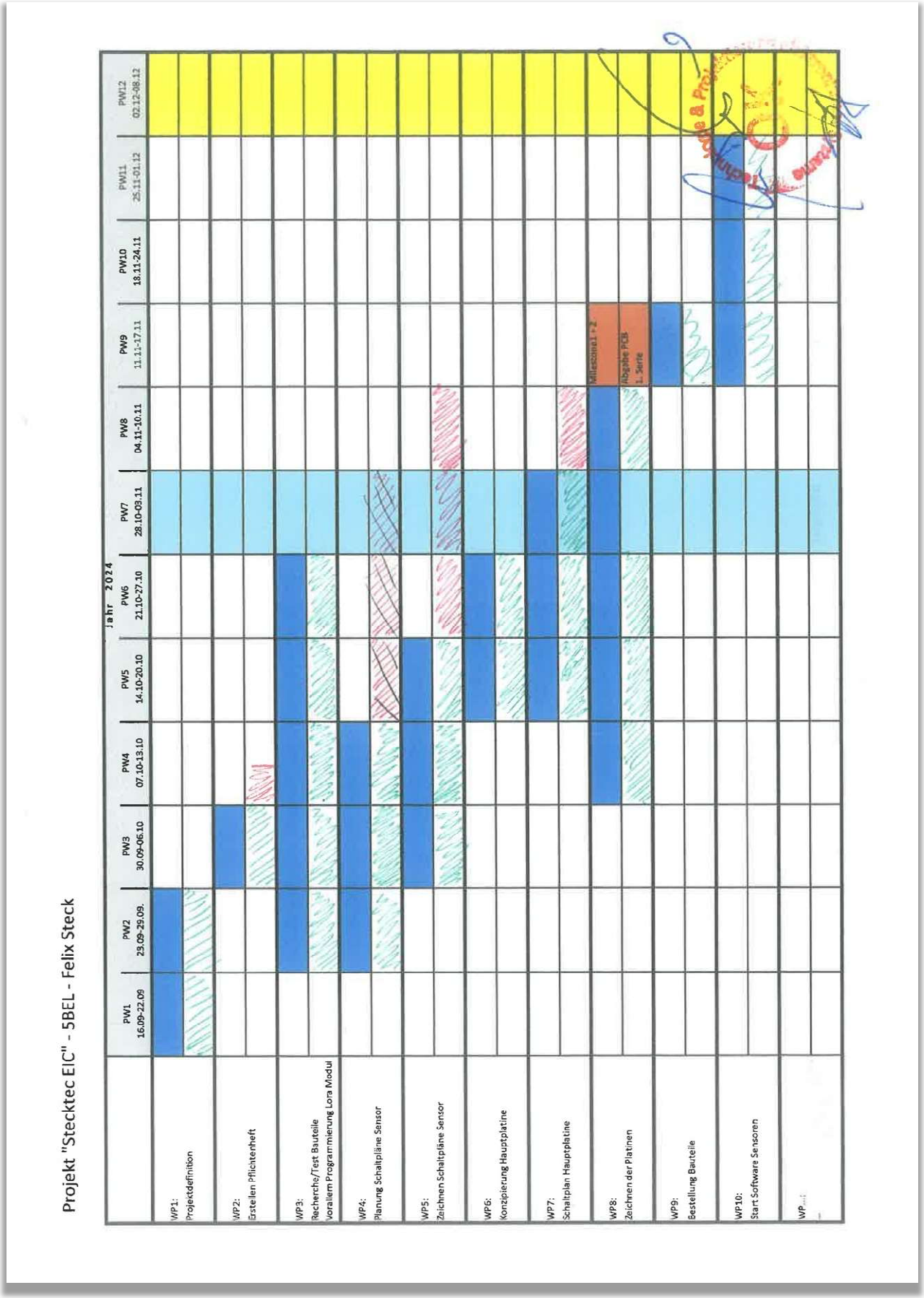
WP20: Alle möglichen auftretenden Bugs ausbessern

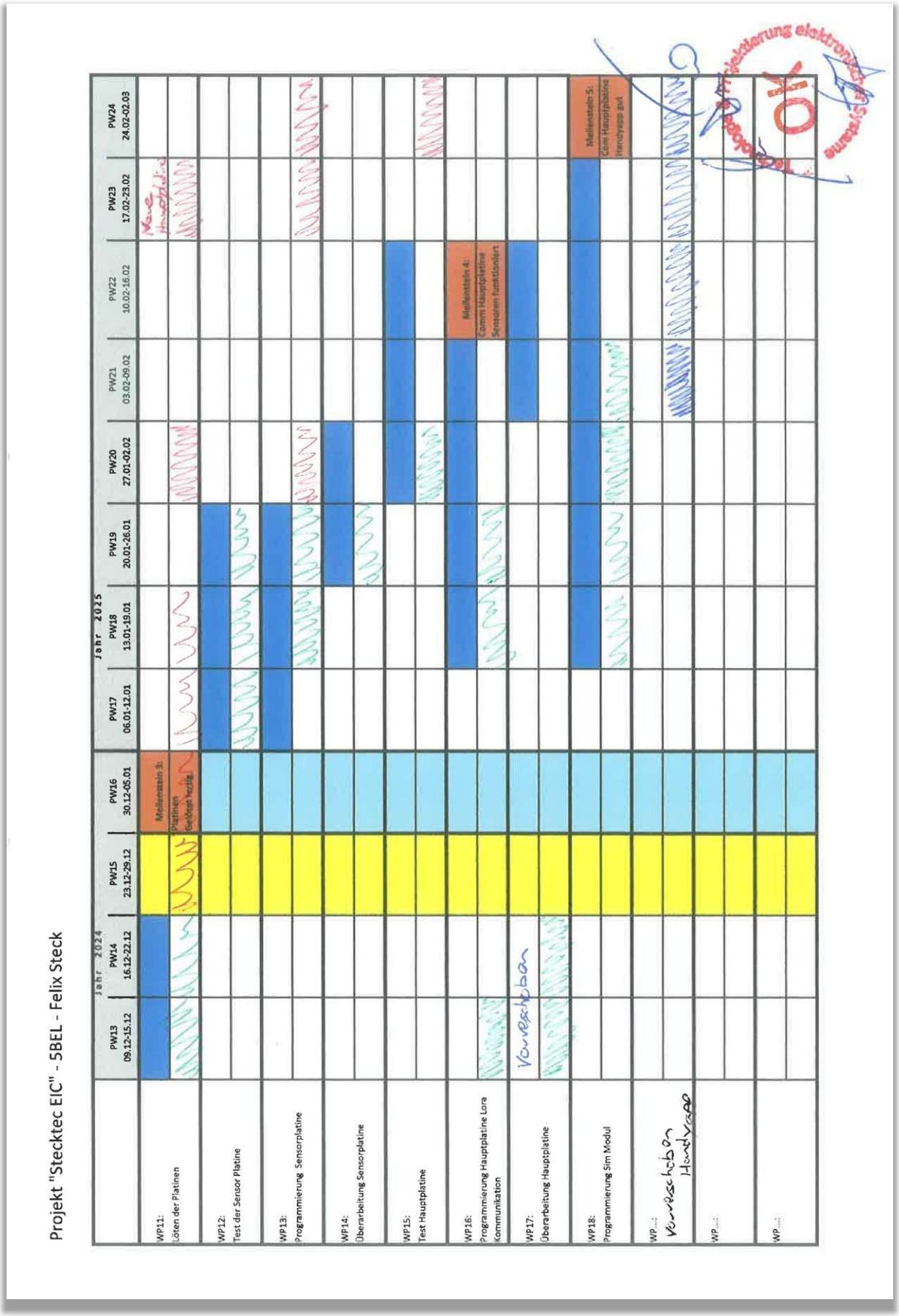
WP21: Projektdokumentation → Komplette Dokumentation schreiben

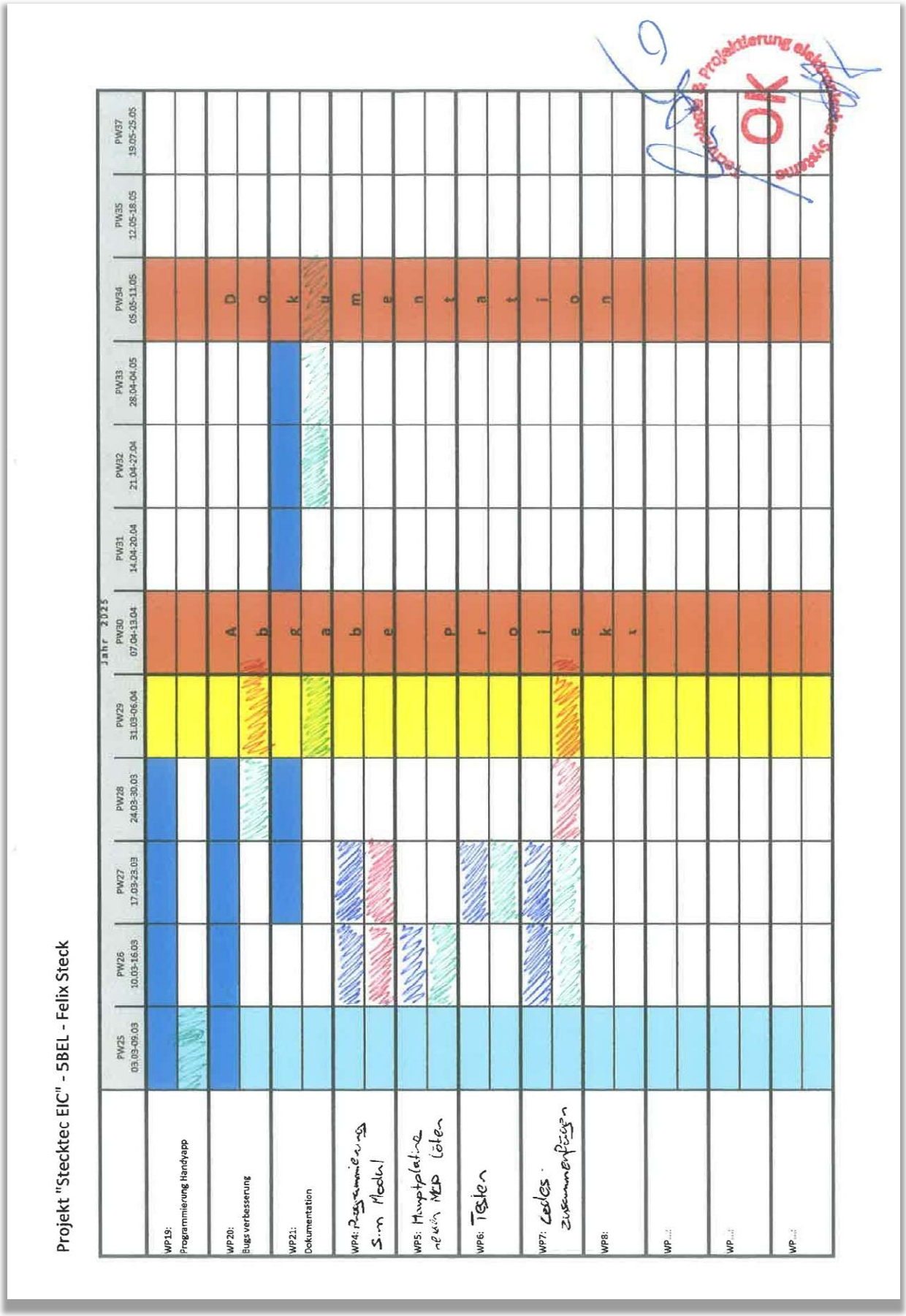
## 7.4 Ganttdiagramm

Ein Gantt-Diagramm ist ein wichtiges Hilfsmittel im Projektmanagement, um den zeitlichen Verlauf eines Projekts übersichtlich darzustellen. Es visualisiert, wann bestimmte Aufgaben beginnen und enden, wie lange sie dauern und in welchem Zusammenhang sie zueinander stehen. Jede Aufgabe wird dabei als horizontaler Balken entlang einer Zeitachse abgebildet – die Länge des Balkens entspricht der jeweiligen Dauer der Aufgabe.

Zu Beginn des Schuljahres erstellte jeder Schüler ein eigenes Gantt-Diagramm für sein Projekt. Dabei lag der Schwerpunkt weniger auf einer detaillierten Planung, sondern vielmehr darauf, ein grundlegendes Verständnis für die zeitliche Struktur, den Arbeitsaufwand und die Verteilung der Aufgaben innerhalb eines Projekts zu entwickeln. So konnten die Schüler erste praktische Erfahrungen in der Projektplanung sammeln.


Projekt "Stecktec EIC" - 5BEL - Felix Steck
Jahr 2024
PW1 16.09-22.09
PW2 23.09-29.09.
PW3 30.09-06.10
PW4 07.10-13.10
PW5 14.10-20.10
PW6 21.10-27.10
PW7 28.10-03.11
PW8 04.11-10.11
PW9 11.11-17.11
PW10 18.11-24.11
PW11 25.11-01.12
PW12 02.12-08.12
WP1: Projektdefinition
WP2: Erstellen Pflichtenheft
WP3: Recherche/Test Bauteile Vorallem Programmierung Lora Modul
WP4: Planung Schaltpläne Sensor
WP5: Zeichnen Schaltpläne Sensor
WP6: Konzipierung Hauptplatine
WP7: Schaltplan Hauptplatine
WP8: Zeichnen der Platinen
WP9: Bestellung Bauteile
WP10: Start Software Sensoren
Meilenstein nr 2
Abgabe PCB 1. Serie


Projekt "Stecktec EIC" - 5BEL - Felix Steck
Jahr 2024
Jahr 2025
PW13 09.12-15.12
PW14 16.12-22.12
PW15 23.12-29.12
PW16 30.12-05.01
PW17 06.01-12.01
PW18 13.01-19.01
PW19 20.01-26.01
PW20 27.01-02.02
PW21 03.02-09.02
PW22 10.02-16.02
PW23 17.02-23.02
PW24 24.02-02.03
WP11: Löten der Platinen
Meilenstein 3: Platinen Gelötet Testig
WP12: Test der Sensor Platine
WP13: Programmierung Sensorplatine
WP14: Überarbeitung Sensorplatine
WP15: Test Hauptplatine
WP16: Programmierung Hauptplatine Lora Kommunikation
Meilenstein 4: Comm Hauptplatine Sensoren funktionieren
WP17: Überarbeitung Hauptplatine
Vorweihnachten
WP18: Programmierung Sim Modul
Meilenstein 5: Com Hauptplatine Handyapp gut
WP ...: Verwiesch hoben Handyapp
WP ...:
WP ...:
Projektierung elektronischer Systeme
OK


Projekt "Stecktec EIC" - 5BEL - Felix Steck
Jahr 2025
PW25 03.03-09.03
PW26 10.03-16.03
PW27 17.03-23.03
PW28 24.03-30.03
PW29 31.03-06.04
PW30 07.04-13.04
PW31 14.04-20.04
PW32 21.04-27.04
PW33 28.04-04.05
PW34 05.05-11.05
PW35 12.05-18.05
PW37 19.05-25.05
WP19: Programmierung Handyapp
WP20: Bugs verbessern
WP21: Dokumentation
WP4: Programmierung Sim Modul
WP5: Hauptplatine neuen MCU löten
WP6: Testen
WP7: Leides Zusammenfügen
WP8:
WP...
WP...
WP...
Abgabe
Dokumentation
Technische Projektierung elektronischer Systeme
OK

## 7.5 Wochenberichte

Im Laufe des Projekts musste jeder Schüler, wöchentlich einen schriftlichen Wochenbericht verfassen. Diese dienten der Reflexion des eigenen Fortschritts, der frühzeitigen Erkennung von Verzögerungen und einer lückenlosen Dokumentation.

Ein solcher Bericht besteht aus mehreren Abschnitten. Zunächst wird kurz beschrieben, welche Arbeiten in der jeweiligen Woche durchgeführt wurden – etwa der Aufbau von Schaltungen, Funktionstests oder Lötarbeiten. Auch auftretende Probleme und deren Lösungen wurden festgehalten.

Anschließend erfolgte ein Abgleich mit dem ursprünglichen Gantt-Diagramm, um eventuelle Abweichungen vom Zeitplan zu erkennen. Falls nötig, wurden Maßnahmen zur Korrektur formuliert.

Im letzten Teil wurde der Zeitaufwand dokumentiert – sowohl die Stunden in der Schule als auch zusätzliche Arbeitszeit zu Hause oder im offenen Labor.

Auf der nächsten Seite ist ein Beispiel Wochenbericht abgebildet.

# Wöchentlicher Projektbericht

STECKTEC

| | | | |
|---|---|---|---|
| Projekttitel: | Stecktec EIC | | |
| Projektleiter: | Felix Steck | | |
| Projektwoche / Datum: | PW: 24 | 24.02.2025 | 02.03.2025 |

Kurzbericht der erledigten Arbeiten in der Projektwoche:

1.5h Programmierung Handyapp Account Managment
1h Programmierung Handyapp Account Managment
3h (Schule) Programmierung Sensorplatine (Auslesen BME280 und über Lora versenden (noch nicht efolgreich))
2h Programmierung Handyapp Wiesen in Firebase speichern
3h Programmierung Handyapp connection mit Firebase

Im Zeitplan laut GANTT-Diagramm: ☐ JA ☒ NEIN

Wenn NEIN, welche Working Packages (WP) liegen in Verzug?

Programmierung Sensorplatine nicht fertig

Meilenstein Kommunikation Hauptplatine Handyapp noch nicht erreicht

Vorgeschlagene Maßnahmen, um den Zeitplan wieder aufzuholen:

Weiter Programmieren (Handyapp und Sensorplatine/Hauptplatine)

Wurden die Maßnahmen aus letzter Woche planmäßig umgesetzt? ☐ JA ☒ NEIN

Zu wenig weiterprogrammiert

Kurze Zusammenfassung des Zeitaufwandes der Projektwoche:

| | | |
|---|---|---|
| Stunden in der Schule (Labor) | 3 | Stunden |
| Stunden im offenen Labor | | Stunden |
| Stunden Privat / Zuhause | 7.5 | Stunden |
| Gesamtstundenzahl | 10.5 | Stunden |

Datum: 03.03.2025

Unterschrift verantwortlicher Projektleiter:

Felix Steck

## 7.6 Kostenrechnung

### 7.6.1 Beraterstunden

Beraterstunden sind Gespräche mit Personen, die spezielles Fachwissen in einem bestimmten Bereich haben und bei einem Projekt weiterhelfen können. Dabei kann es sich um Lehrpersonen oder externe Expertinnen und Experten handeln, welche sich besonders gut mit einem bestimmten Thema auskennen.

Solche Stunden nutzt man, um Fragen zu klären, Probleme zu lösen oder gezielte Unterstützung zu erhalten. Neben der Beratung in Gesprächen kann eine Fachperson in bestimmten Fällen auch konkrete Aufgaben übernehmen – etwa eine Funktion programmieren oder bei der Auswahl von Komponenten helfen.

In der Schule sind solche Unterstützungen meist kostenlos. In echten Projekten hingegen können sie teuer werden: Fachliche Beratung kostet oft rund 100 € pro Stunde. Daher sollte man sich gut vorbereiten und gezielte Fragen stellen, um die Zeit möglichst effizient zu nutzen.

In der Tabelle ist eine Auflistung der von mir in Anspruch genommenen Beraterstunden, insgesamt sind es 6,5 Stunden, was bei einem Stundensatz von 100€, 750€ wären.

| **Beraterstunden** | | | | |
|---|---|---|---|---|
| Datum | Anzahl Stunden | Name Berater | Aufgabe | Kosten (100€/h) |
| 01.10.2024 | 1 | Detomaso Martin | Projektspezifizierung | 100 € |
| 15.10.2024 | 0,5 | Huber Ivan | Auswahl H-Brücke | 50 € |
| 12.11.2024 | 0,5 | Detomaso Martin | Schematic Beratung | 50 € |
| 17.12.2024 | 0,5 | Huber Ivan | Platine Routen | 50 € |
| 12.01.2025 | 1 | Detomaso Martin | Neue Solarplatine | 100 € |
| 13.02.2025 | 1 | Felix Reiter | Programmierung Hilfe | 100 € |
| 25.02.2025 | 0,5 | Detomaso Martin | Programmierung SIM-Modul | 50 € |
| 01.03.2025 | 1,5 | Felix Salcher | Programmierung Handyapp | 150 € |
| 27.03.2025 | 1 | Detomaso Martin | Problem SIM Programm | 100 € |
| | Gesamtstunden | | | Gesamtkosten |
| | 7,5 | | | 750 € |

### 7.6.2 Arbeitsstunden

Die kompletten Arbeitsstunden für das Projekt, die Grauen Wochen beinhalten die Arbeitsstunden für die Projektdokumentation.

| Arbeitsstunden | | | | | |
|---|---|---|---|---|---|
| Woche | zu Hause [h] | Schule [h] | Offenes Labor [h] | Gesamt[h] | Arbeitskosten (40€/h) |
| 1 | 7 | 3 | 0 | 10 | 400 € |
| 2 | 1,5 | 3,5 | 0 | 5 | 200 € |
| 3 | 5 | 5 | 0 | 10 | 400 € |
| 4 | 1 | 4 | 0 | 5 | 200 € |
| 5 | 5,5 | 4,5 | 0 | 10 | 400 € |
| 6 | 0 | 4,5 | 0 | 4,5 | 180 € |
| 7 | 4 | 0 | 0 | 4 | 160 € |
| 8 | 15 | 5 | 0 | 20 | 800 € |
| 9 | 4 | 6 | 0 | 10 | 400 € |
| 10 | 2 | 2 | 0 | 4 | 160 € |
| 11 | 6 | 5 | 0 | 11 | 440 € |
| 12 | 1 | 2 | 1 | 4 | 160 € |
| 13 | 3 | 5 | 0 | 8 | 320 € |
| 14 | 5 | 5 | 0 | 10 | 400 € |
| 15 | 7 | 0 | 0 | 7 | 280 € |
| 16 | 8 | 0 | 0 | 8 | 320 € |
| 17 | 7 | 5 | 2 | 14 | 560 € |
| 18 | 4 | 2 | 0 | 6 | 240 € |
| 19 | 8 | 5 | 0 | 13 | 520 € |
| 20 | 3 | 2 | 0 | 5 | 200 € |
| 21 | 5 | 5 | 0 | 10 | 400 € |
| 22 | 4 | 3 | 0 | 7 | 280 € |
| 23 | 13 | 0 | 0 | 13 | 520 € |
| 24 | 8 | 3 | 0 | 11 | 440 € |
| 25 | 12 | 0 | 0 | 12 | 480 € |
| 26 | 7 | 5 | 0 | 12 | 480 € |
| 27 | 5 | 5 | 0 | 10 | 400 € |
| 28 | 4 | 5 | 0 | 9 | 360 € |
| 29 | 6,5 | 3 | 2 | 11,5 | 460 € |
| 30 | 8 | 5 | 0 | 13 | 520 € |
| 31 | 3 | 0 | 0 | 3 | 120 € |
| 32 | 5 | 0 | 0 | 5 | 200 € |
| 33 | 3 | 0 | 0 | 3 | 120 € |
| 34 | 12 | 0 | 0 | 12 | 480 € |
| 35 | 7 | 0 | 0 | 7 | 280 € |
| | | | | Gesamtstunden | Gesamtarbeitskosten |
| | | | | 307 | 12.280 € |

### 7.6.3 Platinenkosten

Zu Beginn des Projekts wurden die ersten vier Platinen gemeinsam in einer ersten Bestellung am 26.11.2024 bestellt. Nach Fertigzeichnen der Hauptplatine wurde diese ca. eine Woche später auch bestellt. Aufgrund der bereits erwähnten Probleme mit der Hauptplatine und der Solarversorgungsplatine wurden von beiden überarbeitete Versionen entwickelt und nochmals bestellt. Die Gesamtkosten für alle Platinen, einschließlich Lieferung und Steuern, belaufen sich auf 56,74 €. Allerdings muss man dabei beachten, dass bei JLCPCB die Mindestbestellmenge immer 5 Platinen pro Design ist.

| Platinenbestellung 1 | | | | |
|---|---|---|---|---|
| Platine | Platine Kosten | | Bestelldatum: | 26.11.2024 |
| Magnetventilsteuerung | 4,04 € | | Gesamtkosten Platinen: | 17,17 € |
| Programmiergerät | 2,02 € | | Gesamtkosten Lieferung: | 10,00 € |
| Sensorplatine | 7,07 € | | Gesamtkosten Stuern: | 7,00 € |
| Solarversorgung V1 | 4,04 € | | Gesamtkosten Bestellung1: | 34,17 € |

| Gesamtübersicht Platinen Kosten | | | | |
|---|---|---|---|---|
| Platine | Bestelldatum | Platine Kosten | Restliche Kosten | Gesamtkosten |
| Platinenbestellung 1 | 26.11.2024 | 17,17 € | 17,00 € | 34,17 € |
| Hauptplatine V1 | 04.12.2024 | 5,30 € | 3,44 € | 8,74 € |
| Solarversorgung V2 | 08.01.2025 | 2,23 € | 2,54 € | 4,77 € |
| Hauptplatine V2 | 23.01.2025 | 5,30 € | 3,44 € | 8,74 € |
| | | Gesamtkosten Platinen: | | 56,42 € |

### 7.6.4 Bauteilkosten LCSC

Die benötigten elektronischen Bauteile wurden in mehreren Bestellungen bei LCSC beschafft. Inklusive Versand beliefen sich die Gesamtkosten für alle Bauteilbestellungen auf 167,81 €.

| Bauteilbestellungen LCSC | | | | |
|---|---|---|---|---|
| Bestellung | Bestelldatum | Bauteilkosten | Lieferkosten | Gesamtkosten |
| Bestellung 1 | 27.11.2024 | 70,61 € | 5,00 € | 75,61 € |
| Bestellung 2 | 08.01.2025 | 25,60 € | 5,00 € | 30,60 € |
| Bestellung 3 | 28.01.2025 | 56,60 € | 5,00 € | 61,60 € |
| | | Gesamtkosten Bauteile: | | 167,81 € |

### 7.6.5 Kosten Module

Zusätzlich zu den Einzelbauteilen wurden verschiedene fertige Module wie Sensoren, Ventile und Kommunikationsmodule bestellt. Diese Module verursachten Gesamtkosten in Höhe von 199,33 €.

| Bauteilbestellungen LCSC | | | | | |
|---|---|---|---|---|---|
| Bauteil / Modul | Verkäufer | Bestelldatum | Bauteilkosten | Lieferkosten | Gesamtkosten |
| Lora Module (2Stk) | Amazon | 17.09.2024 | 22,18 € | 0,00 € | 22,18 € |
| Lora Antenne (5Stk) | Amazon | 17.09.2024 | 13,00 € | 0,00 € | 13,00 € |
| Durchflussensor (2Stk) | Amazon | 17.09.2024 | 10,00 € | 0,00 € | 10,00 € |
| Soarmodul (5Stk) | Aliexpress | 04.10.2024 | 10,30 € | 0,00 € | 10,30 € |
| Simcom SIM7080 | Aliexpress | 04.10.2024 | 23,00 € | 6,35 € | 29,35 € |
| Simcom SIM7000 | Aliexpress | 13.12.2024 | 33,90 € | 3,70 € | 37,60 € |
| BME280 Sensor | Aliexpress | 26.01.2025 | 3,50 € | 0,00 € | 3,50 € |
| Magnetventil | Aliexpress | 26.01.2025 | 20,40 € | 5,00 € | 25,40 € |
| Irrometer 200SS | Irrometer | 27.01.2025 | 40€ | 8€ | 48€ |
| | | | Gesamtkosten andere Module: | | 199,33 € |

### 7.6.6 Gesamtkosten Projekt

In der folgenden Tabelle sind nochmals alle einzelnen Kostenkategorien übersichtlich aufgelistet. Am Ende wird der Gesamtpreis des Projekts sowohl mit als auch ohne Berücksichtigung der Arbeitsstunden dargestellt.

| Gesamtkosten Projekt | |
|---|---|
| **Kategorie** | **Kosten** |
| Platinenkosten | 56,74 € |
| Bauteilkosten LCSC | 167,81 € |
| Kosten Module | 199,33 € |
| Arbeitskosten | 12.280,00 € |
| Beraterkosten | 750,00 € |
| Gesamtkosten Projekt ohne Arbeitsstunden | 423,88 € |
| Gesamtkosten Projekt mit Arbeitsstunden | 13.453,88 € |

### 7.8.2 Kosten Serienfertigung

Für eine angenommene Serienproduktion von 100 Einheiten eines kompletten Systems (jeweils bestehend aus fünf Platinen, Gehäuse, peripheren Sensoren und Kommunikationsmodulen) ergeben sich folgende geschätzte Kosten. Die Arbeitskosten für Bestückung, Programmierung und Test werden mit einem Stundensatz von 40 €/h und einem Aufwand von 3 Stunden pro Einheit kalkuliert.

Da die SMD-Bauteile bereits auf den Platinen bestückt geliefert werden, reduziert sich der manuelle Aufwand weiter. In einer realen Serienfertigung würde zudem eine Reduzierung von THT-Bauteilen und der verstärkte Einsatz vollautomatischer Bestückung die Kosten pro Einheit weiter senken. Dieser Ansatz bleibt hier eine theoretische Kalkulation.

| Serienfertigung 100Stk. | | | |
|---|---|---|---|
| **Kategorie** | **Stückpreis (ca.)** | **Menge** | **Gesamtkosten** |
| Platinen (5 Platinen je Einheit) | 71,00 € | 100 | 7.100,00 € |
| THT-Bauteile (Handbestückung) | 8,50 € | 100 | 850,00 € |
| Gehäuse & Mechanik | 10,00 € | 100 | 1.000,00 € |
| Programmierung & Test (3 h × 40 €/h) | 120,00 € | 100 | 12.000,00 € |
| SIM-Karte | 20,00 € | 100 | 2.000,00 € |
| SIM7000 Modul | 17,00 € | 100 | 1.700,00 € |
| LoRa-Module (2 Stück) | 16,00 € | 100 | 1.600,00 € |
| BME280 Sensor | 1,50 € | 100 | 150,00 € |
| Bodenfeuchtesensor | 40,00 € | 100 | 4.000,00 € |
| Solarzellen (2Stk) | 2,50 € | 100 | 250,00 € |
| Gesamt | 306,50 € | | 30.650,00 € |

# 8 Fazit

Mit dem EIC-Bewässerungssystem wurde ein praxisnahes und zukunftsorientiertes Projekt umgesetzt, das moderne Technik mit den realen Anforderungen der Landwirtschaft verbindet. Ziel war es, ein flexibles, kostengünstiges und energieeffizientes System zu entwickeln, das sich an unterschiedliche Bedingungen und Betriebsgrößen anpassen lässt. Durch den modularen Aufbau, die einfache Bedienung per Smartphone-App und die Möglichkeit zur Erweiterung bietet das System einen echten Mehrwert für kleine wie große Betriebe.

Besonders motivierend war es, an einer Lösung zu arbeiten, die aus einem konkreten Bedarf entstanden ist und in der Praxis einen echten Nutzen bringen kann. Die Arbeit an diesem Projekt hat nicht nur mein technisches Verständnis vertieft, sondern auch gezeigt, wie wichtig sorgfältige Planung und systematisches Vorgehen bei solchen komplexeren Projekten ist.

Natürlich lief nicht alles exakt nach Plan – einige Funktionen konnten innerhalb des Projektzeitraums nicht vollständig umgesetzt werden. Dennoch habe ich gerade aus diesen Herausforderungen wertvolle Erfahrungen gewonnen, die mir bei zukünftigen Projekten helfen werden, realistischer zu planen und flexibler auf unerwartete Schwierigkeiten zu reagieren.

Das Projekt hat meine Begeisterung für die Entwicklung intelligenter, nachhaltiger Lösungen gestärkt und mir gezeigt, wie viel Potenzial in der Verbindung von Technik und Umwelt steckt. Ich bin überzeugt, dass Systeme wie EIC einen wichtigen Beitrag zur ressourcenschonenden und smarten Landwirtschaft der Zukunft leisten können.

# 9 Abbildungsverzeichniss

**Sofern nicht im folgenden Verzeichnis vermerkt, wurden die verwendeten Bilder von mir selbst erstellt.**

Titelblatt Bild S.0: https://www.schoepf-watersystems.it/media/site/c644a4f7d6-1677570594/schoepf-water-systems-leistungen-tropfberegnung-1462x724-crop-50-50.avif

TFO Meran Logo Alle Seiten: https://www.tfo-meran.it/wp-content/uploads/2011/03/tfo-grafik-ressourcen.png

Atmega Pinout S.8: https://preview.redd.it/bggdd9srui351.png?auto=webp&s=921ec4b761b0465685a39ff3ee45230ead62f877

Simcom SIM7000e S.9: https://ae01.alicdn.com/kf/H31eb36ca52344473bb3ade00587646c18.jpg

Lora Modul S.10: https://camo.githubusercontent.com/ad4dd3b0319f335776305b1199fadf1b9f23e0804c1e7aec15558e0b4982d169/68747470733a2f2f7777772e6d6973636869616e74692e6f72672f77702d636f6e74656e742f75706c6f6164732f323031392f30392f4c6f52615f4533322d54544c2d3130302e6a7067

Lora Operating Modes S.10: https://cdn.hackaday.io/images/3476851560100230477.JPG

BME280 S.11: https://down-my.img.susercontent.com/file/cn-11134207-7r98o-lmlguevlx42s3c

Watermark Sensor S.12: https://www.royaleijkelkamp.com/media/pbtp3lkf/14-27-05-watermark-soil-moisture-sensor-granular-matrix.png?format=webp&width=940&heightratio=0.66&mode=crop

Magnetventil S.12: https://m.media-amazon.com/images/I/61gCEkhwbLL.jpg

MCP23017 S.13: https://www.mouser.it/images/microchiptechnology/lrg/inhouse_mcp23017eso_t.jpg

Lithium Akku Bild S.14: https://cdn03.plentymarkets.com/i9a0e0hd8l6w/item/images/100603/full/KP-18650-2200mAh-3-6V-3-7V-Lithium-Ionen-Akku.jpg

Solarmodul S.14: https://m.media-amazon.com/images/I/61AgKhWJyCL._AC_UF350,350_QL50_.jpg

Durchflussensor S.15: https://mm.digikey.com/Volume0/opasdata/d220001/medias/images/2242/MFG_114991172.jpg

Firebase Logo S.18: https://firebase.google.com/static/images/brand-guidelines/logo-vertical.png

React Native Logo S.19: https://www.ncodetechnologies.com/images/icons/react-native-icn(512x512).png

Bootloader burn Schematic S.34: https://europe1.discourse-cdn.com/arduino/original/4X/4/d/d/4ddb849fe5bf2d8361193ed8f6eb256a8815a7e3.png

Projektmanagment Bild S.45: https://murbl.wordpress.com/wp-content/uploads/2012/10/800px-projektplan.jpg

Creative Commons Bild S.62: https://upload.wikimedia.org/wikipedia/commons/thumb/d/d3/Cc_by-nc_icon.svg/800px-Cc_by-nc_icon.svg.png

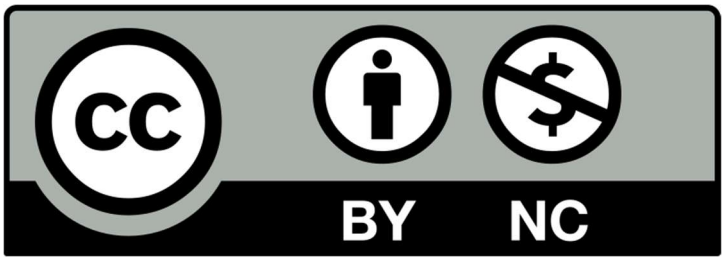
Alle Codausschnitte wurden mit der Website https://chalk.ist/.com erstellt

# 10 Copyright

Dieses Werk ist lizenziert unter einer

Creative Commons – Attribution – NonCommercial Lizenz.

https://creativecommons.org/licenses/by-nc/4.0/



STECKTEC

In dieser Projektdokumentation werden die zentralen Aspekte des Projekts "EIC – Easy Irrigation Control"umfassend dargestellt. Von der ersten Idee über die detaillierte Planung bis hin zur Umsetzung beleuchtet diese Dokumentation alle Phasen dieses innovativen Systems zur automatisierten Bewässerungssteuerung. Das Projekt kombiniert moderne Sensortechnik und drahtlose Kommunikation, um eine effiziente, umweltfreundliche und zukunftsorientierte Lösung für den Obstbau zu schaffen.

Autor: Felix Johann Steck
E-Mail: felix.johann.steck@gmail.com
TEL: +39 344 282 5790

TFO